

Froth

Apuntes de Machine Learning

Aprender construyendo: 47 apuntes nacidos de un proyecto real

De qué es un embedding a por qué 10.921 papers caben en pantalla.
Cada apunte salió de algo que pasó de verdad, tropiezos incluidos.

Proyecto Froth — mapeo semántico de literatura científica

Compilado el 2026-08-16

Contents

Apunte 00 — El mapa del proyecto (léeme primero)	4
Apunte 01 — Python, el motor; y el entorno virtual, la caja de herramientas	6
Apunte 02 — APIs y JSON: cómo se piden datos a un servidor	8
Apunte 03 — Cuando internet dice que no: SSL, códigos de error y API keys	10
Apunte 04 — Tablas de datos: pandas y el archivo parquet	12
Apunte 05 — Qué es un embedding (el corazón del proyecto)	14
Apunte 06 — Cargar SPECTER y tu primer vector (usar vs. construir un modelo)	16
Apunte 07 — Vectorizar los 126 papers: la matriz de embeddings	18
Apunte 08 — Similitud coseno: medir qué tan parecidos son dos papers	20
Apunte 09 — El buscador semántico (juntamos todo)	22
Apunte 10 — Cuadernos Jupyter: tu buscador interactivo	24
Apunte 11 — Tu primera web app con Streamlit	26
Apunte 12 — Reducción de dimensiones: de 768 a 2 sin perder el vecindario	28
Apunte 13 — UMAP en acción: el primer mapa de tus 126 papers	30
Apunte 14 — Clustering: que el algoritmo encuentre los grupos (HDB-SCAN)	32
Apunte 15 — Ponerle nombre a las burbujas: c-TF-IDF	34

Apunte 16 — El bubble map interactivo (Plotly) y el truco del slider	36
Apunte 17 — El filtro de relevancia: el sistema se limpia solo	38
Apunte 18 — Git y GitHub: puntos de guardado y respaldo en la nube	40
Apunte 19 — Grafos: nodos, aristas, hubs y puentes	42
Apunte 20 — La red de similitud: K vecinos, umbral, y tus hubs y puentes reales	44
Apunte 21 — El mapa mental de temas (y el hallazgo de los silos)	46
Apunte 22 — Las vistas finales: red con física y circle packing	48
Apunte 23 — DOI, open access y la lista de lectura	50
Apunte 24 — Qué es un gap (y cómo se mide de verdad)	51
Apunte 25 — De la teoría al texto: <code>find_gaps()</code> y el borrador de review	53
Apunte 26 — El modo Tarea por dentro: trocear y promediar (mean pooling)	55
Apunte 27 — Granularidad recomendada: que los datos elijan (DBCV)	56
Apunte 28 — Conectores multi-fuente: tu cuenta institucional entra al juego	57
Apunte 29 — Fase 6: generalizar no es entrenar (y el motor multi-topic)	59
Apunte 30 — El nacimiento de specter-mineral: tu primer entrenamiento real	61
Apunte 31 — El veredicto del fine-tuning: especialización, no magia	63
Apunte 32 — h-index por cluster: cuántos papers son “imperdibles”	64
Apunte 33 — Resortes calibrados y papers frontera: qué significa la posición	

en la red	66
Apunte 34 — Resumen extractivo vs abstractivo: por qué el nuestro no puede alucinar	67
Apunte 35 — La frase centroide: el filtro de relevancia al revés	68
Apunte 36 — TextRank: PageRank sobre frases (el hub de las frases)	70
Apunte 37 — MMR: relevante pero distinto (y un objetivo que salió degenerado)	72
Apunte 38 — El draft v2: mezclar jueces con Borda y citar por construcción	74
Apunte 39 — El LLM como pulidor verificado (nunca como autor)	76
Apunte 40 — Títulos de cluster con sentido: KeyBERT (no oraciones recordadas)	78
Apunte 41 — El veredicto del re-fine-tune: más datos NO mejoró	80
Apunte 42 — Cuando el programa no se cae sino se CUELGA: watchdogs y subprocessos	82
Apunte 43 — La puerta de relevancia: separar el grano de la paja en 190.000 papers	84
Apunte 44 — Tripletes de citación: entrenar como SPECTER — y el veredicto invertido	87
Apunte 45 — Zoom semántico: por qué 10.921 papers caben en pantalla y 1.772 no	90
Apunte 46 — Por qué un programa tarda en abrir: importar tarde y no calcular dos veces	94

Apunte 00 — El mapa del proyecto (léeme primero)

Proyecto Froth · aprender Machine Learning construyendo

Qué estamos construyendo y por qué

Froth es una herramienta que, dado un tema científico, descarga cientos de papers, los “entiende” por su contenido y dibuja un *mapa de subtemas*: burbujas de papers agrupados por significado, conexiones entre grupos, y zonas vacías (los *gaps* donde puedes aportar algo nuevo en tu tesis).

El proyecto tiene un doble objetivo, y los dos pesan igual:

1. **Aprender Machine Learning de verdad**, pieza a pieza, construyendo.
2. **Tener un producto útil** para literature reviews completos (no el clásico “5 papers que confirman lo que ya pensaba”).

Concepto: pipeline (tubería)

Todo el sistema es una **cadena de 5 pasos** donde la salida de uno es la entrada del siguiente:

[1] PULL → [2] EMBED → [3] REDUCE → [4] CLUSTER → [5] VISUALIZE

1. **Pull**: bajar papers (título, resumen, autores...) desde bases de datos públicas.
2. **Embed**: convertir cada resumen en un *vector* (lista de números que captura su significado).
3. **Reduce**: aplastar esos vectores de ~768 dimensiones a 2 para poder dibujarlos.
4. **Cluster**: agrupar los puntos cercanos = subtemas descubiertos automáticamente.
5. **Visualize**: burbujas, mapa mental, detección de gaps, borrador de review.

Cada paso es, a la vez, un concepto de ML que vas a dominar.

El mapa de carpetas (qué es cada cosa y en qué orden mirar)

Dentro de `Documents\Froth\` está todo. **Regla de oro**: las carpetas *con número* son tuyas (entra); las *sin número* son la maquinaria (no tocar). La chuleta completa está en el archivo `0_LEEME_PRIMERO.md` de la raíz.

Tuyas (numeradas):

1. `1_Apuntes/` — **tu biblioteca** (estos PDFs). Léelos en orden: 00, 01, 02...
2. `2_Datos/` — los datos: `raw/` (crudos), `processed/` (tablas limpias), `embeddings/` (vectores). Se llena solo al correr el pipeline.
3. `3_Resultados/` — las salidas visibles: mapas, reviews.
4. `4_Notebooks/` — cuadernos de experimentación (Jupyter), cuando lleguen.
5. `5_Bitacoras/` — decisiones tomadas y tu diario de aprendizaje.

Maquinaria (sin número, no tocar):

- `froth/` — **el código fuente** (el motor). Un archivo por paso del pipeline: `config.py` (ajustes) → `pull.py` → `embed.py` → `cluster.py` → `graph.py` → `gaps.py` → `visualize.py`.
- `.venv/` — la burbuja con Python y sus librerías (ver Apunte 01).

- **tools/** — el compilador que convierte los apuntes en PDF.

Ojo / error típico

¿Por qué **froth/** **no** lleva número, si dijimos que todo debe estar ordenado? Porque Python importa el código por su nombre (`from froth import pull`) y un nombre que empiece por número rompe el lenguaje. El *orden* del pipeline no vive en el nombre de la carpeta sino dentro del código (`froth/__init__.py`). Los apuntes y tus carpetas, en cambio, sí van numerados porque nadie los importa: solo se leen.

Las 7 fases, de un vistazo

- F0 Cimientos** — entorno Python, Git, estructura. (*casi cerrada*)
- F1 Pull** — bajar 200–500 papers de un topic. (*primera versión ya funciona: 126 papers*)
- F2 Embeddings** — texto → vectores; buscador semántico. (*siguiente*)
- F3 Clusters** — las burbujas de subtemas (UMAP + HDBSCAN + BERTopic).
- F4 Mapa mental** — grafo de relaciones entre papers y temas.
- F5 Gaps + borrador** — zonas vacías del mapa y texto estructurado de review.
- F6 Deep learning** — PyTorch, transformers, afinar tu propio modelo.

Cómo usar estos apuntes

Cada vez que aparezca una habilidad o concepto nuevo en el proyecto, se genera un apunte PDF numerado que se guarda en **apuntes/**. Tú solo entras ahí y lees en orden. Cada apunte tiene cajas de colores: **Concepto** (la idea), **Pruébalo tú** (comandos para teclear tú mismo) y **Ojo** (errores típicos).

En una frase

Froth es una tubería de 5 pasos que convierte “un tema” en “un mapa de la literatura”; este apunte es tu plano general: carpetas, fases y orden de lectura.

Apunte 01 — Python, el motor; y el entorno virtual, la caja de herramientas

Proyecto Froth · aprender Machine Learning construyendo

Python: qué es de verdad

Los archivos `.py` de la carpeta `froth/` son texto plano: recetas escritas. **Python** es el programa que sabe leerlas y ejecutarlas (el “motor”). Por eso el primer paso del proyecto fue instalarlo: sin motor, las recetas son papel mojado.

En tu máquina quedó instalado **Python 3.14**. Cuando escribes `python archivo.py` en una terminal, le estás diciendo: “motor, ejecuta esta receta”.

Concepto: librería (package)

Casi nada se programa desde cero. Una **librería** es código empaquetado que otros escribieron y tú instalas y reutilizas: **requests** (hablar con internet), **pandas** (tablas de datos), etc. Se instalan con la herramienta **pip**, que viene con Python. Piensa en apps que le instalas al motor.

El problema que resuelve el entorno virtual

Imagina dos proyectos en tu PC: uno necesita **pandas** versión 2 y otro la versión 3. Si instalas todo “en el sistema”, se pisan entre sí y algo se rompe. La solución estándar:

Concepto: entorno virtual (venv)

Una **carpeta-burbuja** (en nuestro caso `.venv/` dentro del proyecto) que contiene una copia privada de Python y sus librerías, exclusiva de este proyecto. Lo que instalas dentro no toca nada fuera, y viceversa.

Se creó con: `python -m venv .venv`

Y desde entonces, para ejecutar cualquier cosa del proyecto usamos el Python *de la burbuja*: `.venv\Scripts\python.exe`. Esa ruta rara del principio de cada comando significa exactamente eso: “usa el motor de ESTE proyecto”.

requirements.txt: la lista de la compra

El archivo `requirements.txt` lista las librerías que el proyecto necesita, con un comentario de para qué sirve cada una. Cualquiera (tú en otra PC, un compañero) puede recrear el entorno con una sola orden:

```
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

En Froth las instalamos *por fases*: hoy solo las de la Fase 1 (**requests**, **pandas**, **pyarrow**, **truststore**). Las pesadas de ML llegarán cuando toquen.

Ojo / error típico

Si un comando te dice `ModuleNotFoundError: No module named 'pandas'`, casi siempre significa que ejecutaste con el Python **del sistema** en vez del de la burbuja. Revisa que el comando empiece por `.venv\Scripts\python.exe`.

Pruébalo tú (teclado en mano)

Abre PowerShell (tecla Windows → escribe `powershell` → Enter) y:

1. `cd "C:\Users\you\Documents\Froth"`
2. `.venv\Scripts\python.exe --version` (debe decir 3.14)
3. `.venv\Scripts\python.exe -m pip list` (las librerías de tu burbuja)

En una frase

Python ejecuta el código; el `.venv` es la burbuja privada del proyecto con sus librerías; `requirements.txt` es la lista para reconstruirla donde sea.

Apunte 02 — APIs y JSON: cómo se piden datos a un servidor

Proyecto Froth · aprender Machine Learning construyendo

La idea en 30 segundos

Tu PC (el *cliente*) le manda una petición por internet a otra computadora (el *servidor*) y esta responde con datos. Eso es todo. Una **API** (*Application Programming Interface*) es simplemente la ventanilla oficial que un servicio ofrece para que *programas* (no personas con navegador) le pidan cosas.

En Froth usamos la API de **OpenAlex**, un catálogo gratuito con ~250 millones de trabajos académicos.

Concepto: petición HTTP GET

Una petición se escribe como una URL con **parámetros** después del signo ?:

```
https://api.openalex.org/works?search=lepidolite flotation&per_page=25
```

Se lee: “del catálogo **works**, búscame **lepidolite flotation**, dame 25 por página”. En Python, la librería **requests** construye y envía esto por ti:

```
r = requests.get(url, params={...})
```

Concepto: JSON

El servidor responde en **JSON**: un formato de texto con la misma pinta que los diccionarios y listas de Python (`{"title": "...", "year": 2021}`). Por eso `r.json()` lo convierte directamente en objetos de Python con los que ya puedes trabajar. JSON es *el* idioma de intercambio de datos en internet.

Detalles de ciudadano educado

Las APIs públicas son un recurso compartido. Tres costumbres que nuestro `pull.py` ya practica:

1. **Identifícate** (*polite pool*): mandamos nuestro email en el parámetro `mailto`. OpenAlex da mejor servicio a quien da la cara.
2. **No bombardees**: entre búsqueda y búsqueda esperamos 0.3 s (`time.sleep(0.3)`). Miles de peticiones por segundo tumban servicios.
3. **Pide solo lo que necesitas**: el parámetro `select` lista los campos que queremos (título, año, autores...). Menos datos = más rápido para todos.

Concepto: el truco del abstract invertido

OpenAlex no guarda el resumen como texto corrido sino como un *índice invertido*: un diccionario `{palabra: [posiciones donde aparece]}`. Es una curiosidad real de esta API (lo hacen por temas legales y de eficiencia). Nuestra función `_reconstruct_abstract()` lo vuelve a montar: coloca cada palabra en su posición y las une. Primer ejemplo del proyecto de que **los datos reales vienen “sucios”** y hay que trabajarlos.

Pruébalo tú (teclado en mano)

Pega esta URL en tu navegador (¡una API también responde ahí!) y mira el JSON crudo:
`https://api.openalex.org/works?search=lepidolite%20flotation&per_page=2`
Busca los campos `title`, `publication_year` y `abstract_inverted_index` en la respuesta.

En una frase

Una API es una ventanilla para programas: mandas una URL con parámetros, te responden JSON, y `requests + r.json()` lo convierten en diccionarios de Python.

Apunte 03 — Cuando internet dice que no: SSL, códigos de error y API keys

Proyecto Froth · aprender Machine Learning construyendo

Hoy chocamos con dos errores distintos seguidos. Vale oro saber distinguirlos, porque uno era *de tu máquina* y el otro *del servidor* — y se arreglan de formas opuestas.

Error 1: el candado (SSL / certificados)

Concepto: certificados y HTTPS

Cuando tu PC se conecta a `https://api.openalex.org`, antes de mandar nada verifica el **certificado** del servidor: su carnet de identidad digital, firmado por autoridades de confianza. Así sabes que hablas con OpenAlex de verdad y no con un impostor.

Python trae su propia lista de autoridades de confianza. En tu red (corporativa/con antivirus que inspecciona el tráfico), el certificado que llega no estaba en esa lista → Python se negó: `CERTIFICATE_VERIFY_FAILED`. Tu navegador sí funcionaba porque usa la lista de *Windows*, que sí conoce ese certificado.

Arreglo: la librería `truststore` le dice a Python “usa la lista de Windows”. La activamos una sola vez en `froth/__init__.py` y vale para todo el proyecto.

Error 2: la puerta cerrada (código 503)

Concepto: códigos de estado HTTP

Toda respuesta HTTP trae un número de 3 cifras que resume cómo fue:

- **2xx** — bien (200 OK).
- **4xx** — el error es *tuyo*: 404 no existe, 401 sin permiso, 429 pides demasiado rápido.
- **5xx** — el error es *del servidor*: 500 se rompió por dentro, 503 “no disponible” (saturado o en mantenimiento).

Nuestro 503 venía con un mensaje claro: “búsqueda anónima limitada por carga; usa una API key gratuita”. No había nada que arreglar en el código: era su puerta, cerrada para anónimos.

Ojo / error típico

Regla mental para siempre: **SSL/certificados** → **problema en tu lado** (red, Python, certificados). **5xx** → **problema en el lado de ellos** (esperar, identificarse o cambiar de servicio). Reintentar en bucle sin entender cuál de los dos es, es perder el tiempo.

La API key: tu carnet VIP (y por qué es un secreto)

Concepto: API key

Una **API key** es una cadena de texto que identifica tus peticiones ante el servicio. Con ella OpenAlex te da acceso estable (1 USD/día de uso gratuito, unas 1000 búsquedas — de sobra). Se envía como un parámetro más: `api_key=...`

Como te identifica *a ti*, es un **secreto**: quien la tenga gasta tu cuota en tu nombre. Por eso:

- **Nunca** se escribe dentro del código fuente.
- Vive en el archivo `.openalex_key` en la raíz del proyecto (una sola línea).
- Ese archivo está en `.gitignore`: la lista de cosas que Git (control de versiones) tiene prohibido subir a ningún sitio. Si un día publicas el código en GitHub, tu clave no viaja con él.

`config.py` la lee al arrancar (primero busca la variable de entorno `OPENALEX_API_KEY`; si no, el archivo) y `pull.py` la añade a cada petición.

Pruébalo tú (teclado en mano)

Comprueba tú mismo que la clave está donde debe y fuera del alcance de Git:

1. Abre `.gitignore` (con el Bloc de notas) y localiza la línea `.openalex_key`.
2. En PowerShell, dentro de la carpeta del proyecto: `Get-Content .openalex_key` (verás tu clave — solo tú).

En una frase

SSL falló en tu máquina (arreglado con truststore); el 503 era el servidor saturado rechazando anónimos (arreglado con API key); y una API key es un secreto que vive fuera del código y fuera de Git.

Apunte 04 — Tablas de datos: pandas y el archivo parquet

Proyecto Froth · aprender Machine Learning construyendo

Qué conseguimos hoy

El primer botón del proyecto: **126 papers reales** sobre tu topic, guardados en `data/processed/papers.parquet`. Este apunte explica qué es esa “tabla” y por qué se guarda así.

Concepto: DataFrame (pandas)

pandas es LA librería de tablas en Python, y su objeto estrella es el **DataFrame**: una tabla con filas y columnas, como una hoja de Excel pero manejada con código. La nuestra tiene una fila por paper y estas columnas:

<code>id, doi</code>	identificadores únicos del paper
<code>title, year</code>	título y año
<code>citations</code>	cuántas veces lo han citado
<code>authors</code>	autores (texto “A; B; C”)
<code>abstract</code>	el resumen — la materia prima del ML
<code>query</code>	con qué búsqueda apareció (para depurar)

Con un DataFrame haces en una línea cosas que en Excel serían dolorosas:

- `df.drop_duplicates(subset="id")` quita papers repetidos;
- `df[df["abstract"].str.len() > 20]` filtra los que no tienen resumen;
- `df.sort_values("citations")` ordena por citas.

La limpieza importa más de lo que parece

`pull.py` no guarda lo que llega tal cual. Hace tres limpiezas:

1. **Deduplicar**: el mismo paper puede aparecer en varias búsquedas (“lepidolite flotation” y “lithium mica flotation” traen solapados).
2. **Filtrar sin-abstract**: sin resumen no hay significado que vectorizar.
3. **Reconstruir el abstract** (ver Apunte 02).

Ojo / error típico

En nuestros 126 papers se colaron algunos **off-topic** (p. ej. uno de telecomunicaciones). ¿Por qué? Colisión de siglas: buscamos “LCT” (un tipo de pegmatita) y también existe en otros campos. No lo arreglamos a mano: en las Fases 2–3 el propio ML los apartará, porque sus *vectores* quedarán lejos de los de flotación. Los datos reales siempre traen ruido; el sistema debe tolerarlo.

Concepto: parquet vs. CSV

Podíamos guardar la tabla como `.csv` (texto plano separado por comas). Elegimos **parquet** porque:

- **Comprime:** ocupa varias veces menos.
- **Guarda por columnas:** leer solo `year` y `citations` de 100 000 papers no obliga a leer los abstracts.
- **Conserva los tipos:** en CSV todo vuelve como texto y hay que re-adivinar qué era número; parquet recuerda que `year` es un entero.

Es el formato estándar en data science moderno. La librería `pyarrow` es quien sabe leerlo/escribirlo; pandas la usa por debajo con `df.to_parquet()` y `pd.read_parquet()`.

Pruébalo tú (teclado en mano)

Mira tu botón con tus propias manos. En PowerShell, dentro de la carpeta del proyecto, abre un Python interactivo con `.venv\Scripts\python.exe` y escribe línea a línea:

1. `import pandas as pd`
2. `df = pd.read_parquet("data/processed/papers.parquet")`
3. `len(df)` (¿cuántos papers?)
4. `df.columns` (las columnas)
5. `df.sort_values("citations", ascending=False).head(3)[["title", "year"]]`
6. `exit()` para salir.

Acabas de hacer tu primer análisis de datos en pandas.

En una frase

Un DataFrame es una tabla programable; limpiamos duplicados y sin-abstract; y parquet es el formato comprimido, por columnas y con tipos que usa el data science serio.

Apunte 05 — Qué es un embedding (el corazón del proyecto)

Proyecto Froth · aprender Machine Learning construyendo

El problema: la computadora no entiende palabras

Tú, como experto, sabes que estas dos frases hablan de lo mismo:

- “*flotación de partículas finas*”
- “*recuperación de granos minerales pequeños mediante espuma*”

Pero no comparten casi ninguna palabra. Si buscáramos “coincidencia de palabras”, el sistema diría que no tienen nada que ver. Y al revés: “banco de peces” y “banco de dinero” comparten la palabra “banco” y no se parecen en nada.

Conclusión: contar palabras iguales no captura el *significado*. Necesitamos otra cosa.

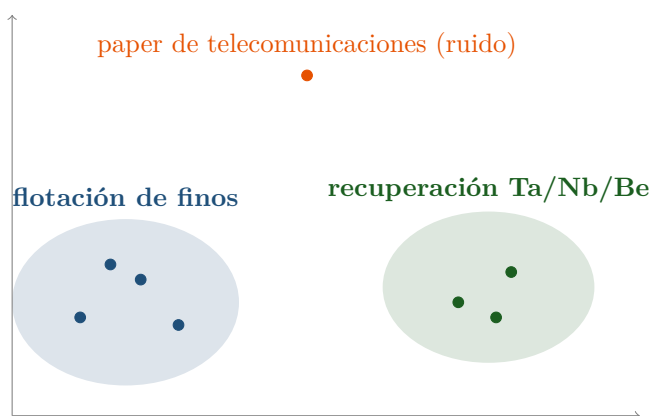
La solución: convertir el texto en un punto en un mapa

Concepto: embedding (vector de significado)

Un **embedding** es un texto convertido en una lista de números —sus *coordenadas*— de modo que se puede colocar como un **punto** en un “mapa del significado”. La lista de números también se llama **vector**.

La propiedad mágica: el mapa está construido para que **textos con significado parecido queden cerca**, y los distintos, lejos. Dos papers que estudian lo mismo caen juntos aunque usen palabras diferentes.

Imagina el mapa de significado de tus papers (aquí en 2D para poder dibujarlo):



Cada punto es un paper. Los de flotación se agrupan a la izquierda; los de by-products, a la derecha; y el paper off-topic que se coló (¿recuerdas el de telecomunicaciones?) queda *solo, lejos de todo* — por eso luego el sistema podrá apartarlo automáticamente.

¿Por qué 768 coordenadas y no 2?

Concepto: dimensiones

En un mapa de verdad usas **2** coordenadas: latitud y longitud. Pero el significado es mucho más rico que una posición en un plano, así que hace falta más “espacio”. El modelo que usaremos (SPECTER) describe cada texto con **~768 coordenadas**.

No lo puedes dibujar (nadie ve en 768 dimensiones), pero eso no importa: las **matemáticas** sí saben medir qué tan cerca están dos puntos aunque tengan 768 coordenadas. En el Apunte 06 veremos cómo (la “similitud coseno”). En la Fase 3 aplastaremos esas 768 a 2 justo para poder dibujar el mapa de verdad.

Cada uno de esos 768 números captura algún “aspecto” abstracto del texto. No son interpretables uno a uno (el número 348 no significa “cuánto habla de litio”); lo que importa es el *patrón completo* de los 768 juntos.

¿De dónde salen los números? Un modelo preentrenado

Concepto: modelo preentrenado

Los 768 números los produce **SPECTER**: una *red neuronal* que la organización AllenAI ya entrenó leyendo millones de papers científicos (y quién cita a quién). Aprendió a colocar textos parecidos cerca.

Lo clave: nosotros **no entrenamos nada**. Descargamos el modelo ya hecho y lo usamos, como quien usa una calculadora sin fabricarla. Eso es un *modelo preentrenado*, y es la forma más rápida y potente de empezar en ML de texto.

Ojo / error típico

¿Por qué SPECTER y no un modelo genérico? Porque está entrenado **con papers**. Un modelo generalista entiende texto común; SPECTER entiende el lenguaje académico (métodos, resultados, jerga científica) mucho mejor para nuestro caso. Herramienta especializada para un trabajo especializado.

Qué haremos con esto (lo que viene)

1. Instalar las librerías que cargan SPECTER (siguiente paso).
2. Convertir tus 126 abstracts en 126 vectores (una matriz 126×768).
3. Medir parecidos con la similitud coseno (Apunte 06).
4. Construir el **buscador semántico**: escribes una frase → te da los 5 papers más parecidos por significado. (El momento “wow”).

En una frase

Un embedding convierte cada texto en un punto (768 coordenadas) dentro de un “mapa del significado” donde lo parecido queda cerca; esos números los da SPECTER, un modelo ya entrenado con papers que nosotros solo reutilizamos.

Apunte 06 — Cargar SPECTER y tu primer vector (usar vs. construir un modelo)

Proyecto Froth · aprender Machine Learning construyendo

Qué hicimos

En el Apunte 05 vimos la *idea* del embedding. Aquí lo tocamos de verdad: tomamos un paper real de tu tabla y lo convertimos en su vector.

1. Cargamos un paper (título + abstract) desde `2_Datos/processed/papers.parquet`.
2. Cargamos el modelo con una línea: `SentenceTransformer("allenai-specter")`.
3. Lo convertimos: `model.encode(texto)` → un vector de **768 números**.

El paper de prueba fue “*Electrostatically Controlled Enrichment of Lepidolite via Flotation*” y su vector empezaba por `[-0.59, -0.16, 1.34, ...]`. Ese paper es ahora un punto en el mapa del significado.

Concepto: modelo preentrenado, en la práctica

La primera vez que llamas a `SentenceTransformer("allenai-specter")`, la librería **descarga** el modelo (~440 MB) desde el repositorio Hugging Face y lo guarda en una *caché* en tu disco. Las siguientes veces lo carga de ahí, al instante, sin internet. No entrenas nada: usas pesos que otros ya calcularon.

Ojo / error típico

Título + abstract, no solo abstract. SPECTER fue entrenado para recibir el título pegado al abstract, porque el título aporta señal fuerte del tema. Por eso el código hace `texto = titulo + ". " + abstract` antes de vectorizar. Un detalle pequeño que mejora los resultados.

Ojo / error típico

Los *warnings* que salieron son inofensivos. Uno hablaba de “symlinks” (una optimización de disco en Windows) y otro de “HF_TOKEN” (una llave opcional para descargar más rápido). Ninguno impide nada: el modelo cargó y funcionó. Aprende a distinguir un *warning* (aviso, sigue adelante) de un *error* (se detiene).

La pregunta clave: ¿construimos nosotros el modelo?

No. Y conviene tener clarísima la diferencia entre dos cosas que suenan parecidas:

	USAR un modelo	ENTRENAR un modelo
¿Qué haces?	Descargas uno ya hecho y le pasas texto	Le enseñas desde cero con datos; ajustas sus “pesos”
¿Cuánto cuesta?	Una línea de código, segundos	Semanas, muchos datos, tarjetas gráficas (GPU)
¿En Froth?	Fases 2 a 5 (lo de ahora)	Fase 6 , opcional y avanzada

Concepto: por qué reutilizar es lo correcto (ahora)

Para dibujar las burbujas y el mapa, el SPECTER genérico —entrenado con millones de papers de todas las áreas— es más que suficiente. Entrenar tu propio modelo de embeddings desde cero sería reinventar la rueda, carísimo y sin ganancia clara en esta etapa. Regla del proyecto: *reciclar sin vergüenza*. Lo original de tu trabajo está en **cómo combinas** las piezas y el problema que resuelves, no en re-fabricar el motor.

Entonces, ¿cuándo sí tocamos “el modelo”?

En la **Fase 6** (la última, opcional). Ahí el experimento con potencial de tesis es *afinar* (fine-tuning) un modelo con papers de tu propio dominio (minerales, flotación, metalurgia) y medir si produce **mejores** clusters que el SPECTER genérico. Eso sí es “construir algo tuyo”, y para entonces habrás aprendido PyTorch y cómo funciona un modelo por dentro. Es el postre, no el plato principal.

En una frase

El modelo de vectores (SPECTER) ya está construido: solo lo descargamos y usamos, no lo entrenamos. Usar $\neq$ entrenar; entrenar lo tuyo es la Fase 6, opcional. Por ahora, tranquilidad: el motor ya está hecho.

Apunte 07 — Vectorizar los 126 papers: la matriz de embeddings

Proyecto Froth · aprender Machine Learning construyendo

De uno a todos

En el Apunte 06 convertimos *un* paper en su vector. Aquí convertimos los **126 de golpe** y **guardamos** el resultado en disco. Esta es la diferencia clave: antes solo mirábamos; ahora producimos un dato que las siguientes fases reutilizarán.

Concepto: matriz (N filas $\times$ M columnas)

Un vector es una fila de números. Si apilas muchos vectores, obtienes una **matriz**: una tabla de números. La nuestra tiene forma (126, 768):

- **126 filas** — una por paper.
- **768 columnas** — las coordenadas de cada paper.

La *fila* i de la matriz es el vector del *paper* i de la tabla: mismo orden. Esa alineación es sagrada — así sabemos qué vector pertenece a qué paper.

Procesar en lote (batch): por qué es más rápido

El código no llama al modelo 126 veces (una por paper). Le pasa la **lista completa** de textos y el modelo los procesa en *lotes* (viste la barra **Batches**: 4/4).

Concepto: batch (lote)

Procesar en lote significa dar muchos ejemplos juntos para que el modelo aproveche mejor el procesador (hace las cuentas en paralelo). Uno por uno funcionaría, pero sería más lento. Nuestros 126 papers tardaron ~ 45 segundos en CPU; con una tarjeta gráfica sería aún más rápido, pero para este tamaño no hace falta.

Guardar en disco: el archivo .npy

Vectorizar cuesta segundos; no queremos repetirlo cada vez. Por eso guardamos la matriz en `2_Datos/embeddings/vectors.npy`.

Concepto: formato .npy (NumPy)

`.npy` es el formato propio de NumPy para guardar arrays/matrices de números tal cual, sin convertir a texto. Se escribe con `np.save()` y se lee al instante con `np.load()`. Nuestra matriz de 126×768 pesa apenas ~ 378 KB. Las fases siguientes (reducir a 2D, agrupar) leerán este archivo en vez de recalcular.

Ojo / error típico

Regla de oro del pipeline: cada fase deja su resultado guardado y la siguiente lo lee. Papers → `papers.parquet`; vectores → `vectors.npy`. Si algo se rompe más adelante, no tienes que volver a bajar papers ni recalculando embeddings: retomas desde el último archivo guardado.

Lo que quedó construido

- La función `embed_papers(df)` en `froth/embed.py` (ya no está vacía).
- El archivo `2_Datos/embeddings/vectors.npy` con la matriz 126×768 .
- Utilidades `save_embeddings()` y `load_embeddings()` para las fases que vienen.

En una frase

Apilamos los 126 vectores en una matriz (126×768), la calculamos en lote (rápido) y la guardamos en `vectors.npy` para no recalcularlo; la fila *i* siempre es el paper *i*.

Apunte 08 — Similitud coseno: medir qué tan parecidos son dos papers

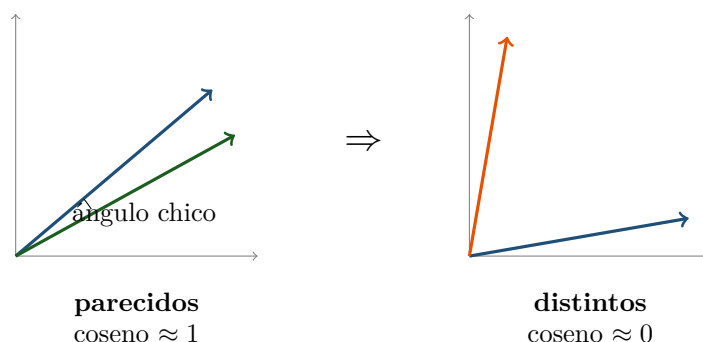
Proyecto Froth · aprender Machine Learning construyendo

El problema

Ya tienes 126 puntos (vectores) en el mapa del significado. Pregunta natural: dados dos papers, ¿cómo pone la computadora un número a “qué tan parecidos son”? Con la **similitud coseno**.

La idea: el ángulo entre dos flechas

Cada vector es una **flecha** que sale del origen y apunta hacia la posición del paper. Para comparar dos papers miramos el **ángulo** entre sus flechas:



Concepto: similitud coseno

Es el **coseno del ángulo** entre dos vectores. Da un número fácil de leer:

- Flechas casi alineadas (ángulo pequeño) → coseno **cercano a 1** → mismo tema.
- Flechas perpendiculares (90°) → coseno **cercano a 0** → sin relación.

Para embeddings de texto el valor suele caer entre 0 y 1.

Ojo / error típico

¿Por qué el *ángulo* y no la *distancia* entre las puntas? Porque lo que importa es **hacia dónde apunta** el significado, no qué tan larga es la flecha. Dos papers del mismo tema, uno con abstract largo y otro corto, tendrán flechas de distinto “tamaño” pero apuntando al mismo lado: el ángulo los reconoce como iguales; la distancia recta se dejaría engañar por el tamaño.

La prueba con tus papers (números reales)

Tomamos el paper “*Enrichment of Lepidolite via Flotation*” y calculamos su coseno contra los otros 125. Esto salió:

Coseno	Paper	¿Tiene sentido?
0.942	Selective Inhibition ... Flotation of Lepidolite	sí: mismo tema
0.941	Froth Flotation of Lepidolite — A Review	sí: mismo tema
0.932	Flotation Collectors for Lepidolite Mineral	sí: mismo tema
0.511	...Radio Resources for Multicell Mobile	no: telecomunicaciones
0.507	Adrienne Rich and the Poetry...	no: poesía
0.495	Theodore Beza... Peace in France 1572	no: historia

Los papers de flotación de lepidolita salieron con coseno ~ 0.94 (casi idénticos en tema), y el ruido off-topic que se coló en la Fase 1 quedó en ~ 0.50 (lejos). El sistema los ordenó bien **sin saber nada de minería**: solo por el significado de los abstracts.

Concepto: la herramienta: `cosine_similarity` de scikit-learn

No calculamos el ángulo a mano. `sklearn.metrics.pairwise.cosine_similarity` toma vectores y devuelve los cosenos. Le pasamos un paper contra la matriz de los 126 y obtuvimos las 126 similitudes de un golpe.

Para qué sirve esto

Este número es el ladrillo del **buscador semántico** (paso siguiente): para encontrar los papers más parecidos a una frase, la convertimos en vector y nos quedamos con los de *mayor* coseno. Y en la Fase 3, la misma idea de “quién está cerca de quién” es la que forma las burbujas de subtemas.

En una frase

La similitud coseno mide el ángulo entre dos vectores: ≈ 1 mismo tema, ≈ 0 sin relación. Con tus papers, flotación-vs-flotación dio 0.94 y flotación-vs-ruido 0.50: funciona.

Apunte 09 — El buscador semántico (juntamos todo)

Proyecto Froth · aprender Machine Learning construyendo

Qué construimos

Una herramienta que recibe una **frase en lenguaje natural** y devuelve los papers más parecidos *por significado* — no por palabras compartidas. Es la función `most_similar()` de `froth/embed.py`, y une todo lo aprendido en la Fase 2.

Concepto: búsqueda semántica vs. búsqueda por palabras

La búsqueda clásica (Ctrl+F, Google básico) empareja **palabras**: si buscas “grain size” no encuentra un texto que diga “coarse and fine particles”, aunque signifiquen lo mismo. La búsqueda **semántica** empareja **significado**: convierte tu frase en un vector y busca los vectores cercanos, así que sí los relaciona.

Cómo funciona por dentro (3 pasos)

1. **Vectorizar la frase.** Se pasa tu frase por el *mismo* modelo (SPECTER) con `model.encode([query])`. Clave: la frase y los papers deben vivir en el mismo mapa, o comparar no tendría sentido.
2. **Medir contra todos.** Con `cosine_similarity` calculamos el coseno de tu frase contra los 126 vectores de golpe.
3. **Quedarnos con los mejores.** Ordenamos de mayor a menor coseno y devolvemos los `k` primeros (por defecto 5), como una tabla `score, title, year`.

Ojo / error típico

El mismo modelo para las dos cosas. Si vectorizaras los papers con SPECTER y la frase con otro modelo distinto, los dos “mapas” no coincidirían y los resultados serían basura. Consistencia del embedding = regla de oro.

La prueba (resultados reales)

Búsqueda: “*how does grain size affect the recovery of minerals by froth flotation*”

Score	Paper
0.873	Role of Bubble Size in Flotation of Coarse and Fine Particles
0.856	Effect of Clay Slime on the Froth Stability and Flotation Performance
0.855	Spodumene Flotation Mechanism

Escribimos “grain size” y encontró papers de “coarse and fine particles” y “ultrafine”: **entendió el significado, no las palabras**.

Búsqueda: “*extracting beryllium and tantalum as valuable by-products*”

Score	Paper
0.812	Niobium and tantalum
0.740	Concentration of Tantalum and Niobium

Trajo justo los papers de los by-products Ta/Nb/Be de tu tesis.

Concepto: score entre 0 y 1

El **score** es la similitud coseno (Apunte 08). Aquí ronda 0.7–0.9: alto, tiene sentido. Ojo: los scores de “frase libre vs. paper” suelen ser algo más bajos que “paper vs. paper” (0.94 antes), porque una frase corta es menos rica que un abstract entero. Lo que importa es el *orden*, no el número absoluto.

Qué significa esto para el proyecto

Ya tienes una herramienta útil de verdad: interrogar tu corpus en lenguaje natural. Y más importante: la maquinaria (vectorizar + coseno + ordenar) es **la misma** que en la Fase 3 formará las burbujas de subtemas. Dominar esto es dominar el núcleo de Froth.

En una frase

El buscador semántico convierte tu frase en vector, la compara por coseno contra los 126 papers y devuelve los más parecidos por significado. Probado: “grain size” encontró “fine/coarse particles”. El motor de Froth ya late.

Apunte 10 — Cuadernos Jupyter: tu buscador interactivo

Proyecto Froth · aprender Machine Learning construyendo

Qué es un notebook y por qué importa

Concepto: Jupyter notebook

Un **notebook** (archivo `.ipynb`) es un documento hecho de **celdas** que puedes ejecutar una a una. Hay dos tipos de celda: de **código** (Python que corre) y de **texto** (explicaciones en Markdown). Ejecutas una celda con **Shift + Enter**.

La gracia: pruebas, ves el resultado justo debajo, cambias algo y vuelves a correr — sin reiniciar todo el programa. Por eso es **la** herramienta de exploración en data science y ciencia de datos. Además, un notebook bien hecho es una pieza excelente de portafolio: cuenta una historia (texto + código + resultados) que cualquiera puede seguir.

Tu cuaderno: `4_Notebooks/01_buscador_semantico.ipynb`

Envuelve la función `most_similar()` en una interfaz cómoda:

1. Una celda de **preparación** carga los 126 papers y sus vectores.
2. Una celda de **consulta** donde cambias la variable `my_query` por la frase que quieras y ejecutas.
3. El resultado aparece como **tabla** (score, título, año, citas), porque `most_similar` devuelve un DataFrame y Jupyter los dibuja bonitos.

Pruébalo tú (teclado en mano)

Ábrelo tú y valida como experto. En PowerShell, dentro de la carpeta del proyecto:

```
1. cd "C:\Users\you\Documents\Froth"
```

```
2. .venv\Scripts\python.exe -m jupyter notebook "4_Notebooks\01_buscador_semantico.ipynb"
```

Se abre en tu navegador. Ejecuta las celdas con Shift+Enter, cambia `my_query` y prueba frases de tu campo. **Tú juzgas** si los papers que salen son los correctos: esa es la validación que cierra la Fase 2.

Ojo / error típico

El notebook empieza con `sys.path.insert(...)` apuntando a la carpeta del proyecto. Es para que Python encuentre el paquete `froth` aunque el cuaderno viva en la subcarpeta `4_Notebooks/`. Sin esa línea, `from froth import embed` fallaría.

Notebook vs. código (.py): ¿cuándo cada uno?

Notebook (.ipynb)	Módulo (.py)
Explorar, probar ideas, enseñar, presentar resultados. Se lee de arriba a abajo como una historia.	Código estable que se reutiliza (el “motor”, como <code>embed.py</code>). Se importa desde otros archivos.

Regla sana: exploras en el notebook; cuando algo funciona y lo vas a reutilizar, lo mueves a un `.py` del paquete. Tu buscador ya vive en `embed.py`; el notebook solo lo *usa*.

En una frase

Un notebook es un documento de celdas ejecutables (Shift+Enter) ideal para explorar y presentar. El tuyo envuelve `most_similar` para que escribas consultas y veas los papers en tabla: úsalo para validar la Fase 2 con tu criterio de experto.

Apunte 11 — Tu primera web app con Streamlit

Proyecto Froth · aprender Machine Learning construyendo

Qué construimos

Una **página web que corre en tu propia PC**: una caja de búsqueda donde escribes una frase y ves los papers más relevantes en tabla. Es el archivo `app.py` en la raíz del proyecto, y por dentro usa el *mismo* `most_similar()` de la Fase 2. Cero motor nuevo: solo una cara bonita encima.

Concepto: Streamlit

Streamlit es una librería que convierte un script de Python en una web interactiva sin que tengas que saber HTML, CSS ni JavaScript. Escribes `st.title(...)`, `st.text_input(...)`, `st.dataframe(...)` y Streamlit dibuja la página. Cada vez que el usuario cambia algo (escribe, mueve el slider), Streamlit **re-ejecuta el script** de arriba a abajo y actualiza la vista.

Cómo arrancarla

Pruébalo tú (teclado en mano)

Desde la carpeta del proyecto, en PowerShell:

1. `cd "C:\Users\you\Documents\Froth"`
2. `.venv\Scripts\python.exe -m streamlit run app.py`

Se abre sola una pestaña en `http://localhost:8501`. Escribe una consulta, mueve el slider de resultados y mira la tabla. Para apagarla: **Ctrl + C** en PowerShell.

Ojo / error típico

“localhost” significa “esta misma máquina”. La web NO está en internet: vive y corre en tu PC. Por eso no pagas dominio ni hosting. Para que otros la usen: o descargan el repo y la corren igual, o la subes gratis a Streamlit Community Cloud / Hugging Face Spaces (ver más abajo).

Dos trucos del código que vale la pena entender

Concepto: caché: no recargar lo pesado en cada clic

Como Streamlit re-ejecuta el script en cada interacción, cargar los 126 papers y sus vectores cada vez sería lento. El decorador `@st.cache_resource` sobre la función `load_data()` hace que se carguen **una sola vez** y se reutilicen. El modelo SPECTER se cachea igual (variable global en `embed.py`).

Además, `app.py` importa `from froth import config, embed`: la web es solo un cliente del motor que ya tienes. Si mañana mejoras `most_similar`, la web mejora sola.

Cómo se comparte (tu meta de portafolio)

- **Descargar y correr:** subes el repo a GitHub; cualquiera hace `git clone`, instala `requirements.txt` y lo corre en su PC. Gratis. (GitHub guarda el código, no lo ejecuta por el visitante.)
- **Link público gratis (opcional):** Streamlit Community Cloud o Hugging Face Spaces hospedan la app gratis y dan una URL pública. Sin dominio de pago.

Lo que viene (visión)

Esta app es el **Módulo Tesis**: entrada = un título/frase. Más adelante se añade el **Módulo Tarea/Investigación**: adjuntas PDFs de instrucciones + escribes lo que te piden, y el mismo motor te devuelve los papers relevantes. Y en la Fase 3, además de la lista, aquí mismo aparecerá el **bubble map** de subtemas.

En una frase

Streamlit convierte tu script en una web local (localhost) sin saber HTML. `app.py` envuelve `most_similar` en una caja de búsqueda; se arranca con `streamlit run app.py`, se comparte por GitHub o hosting gratis, y es la base de los dos módulos futuros.

Apunte 12 — Reducción de dimensiones: de 768 a 2 sin perder el vecindario

Proyecto Froth · aprender Machine Learning construyendo

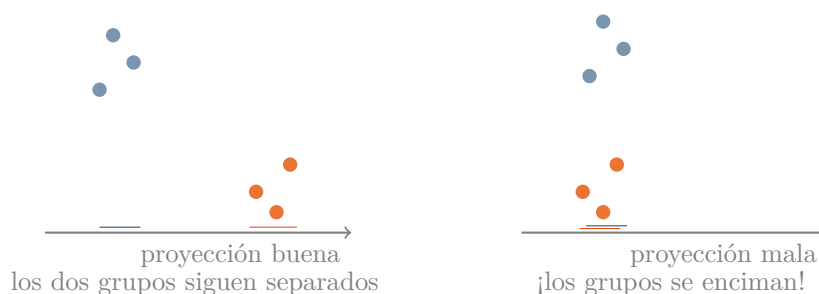
El problema

Cada paper es un punto en **768 dimensiones** (Apuntes 05–07). Tu pantalla tiene **2**. Para dibujar el mapa hay que “aplastar” — pero un mal aplastado encima puntos que no tienen nada que ver, y el mapa engaña. La pregunta de este apunte: *¿cómo se aplasta conservando quién está cerca de quién?*

La intuición clásica: PCA y las sombras

Concepto: PCA (análisis de componentes principales)

Piensa en la **sombra** de tu mano en la pared: un objeto 3D proyectado en 2D. Si orientas bien la mano, la sombra conserva la forma (se ven los dedos); si la orientas mal, todo se encima. **PCA busca el “ángulo de luz” que menos información pierde:** las direcciones en las que los datos más varían. Es la técnica clásica, rápida y muy usada.



Limitación: PCA solo hace proyecciones *rectas* (lineales). Si los datos forman nubes curvas o enredadas —como suele pasar con texto—, hasta el mejor ángulo encima cosas.

La moderna: UMAP y la mudanza de vecindarios

Concepto: UMAP

UMAP no proyecta sombras. Trabaja en dos tiempos:

1. En 768D, apunta para cada paper **quiénes son sus vecinos más cercanos**.
2. Coloca los puntos en 2D doblando y estirando lo que haga falta, con una sola obsesión: **que cada punto conserve a sus vecinos al lado**.

Como mudar una ciudad a un mapa más pequeño garantizando que cada casa siga junto a las mismas casas de antes, aunque las distancias largas cambien. Por eso UMAP separa nubes que PCA encima: no está limitado a proyecciones rectas.

La letra pequeña: todo mapa 2D miente un poco

Ojo / error típico

Al aplastar 768 dimensiones a 2 **siempre** se pierde algo. En un mapa UMAP:

- **Confía en:** quién está pegado a quién; las nubes/grupos que se forman.
- **NO confíes en:** las distancias largas (“este cluster está al doble de distancia que aquel”) ni en el tamaño aparente de las nubes.
- Los ejes X e Y **no significan nada** (no son “más flotación” o “más litio”); son solo el resultado del acomodo. Por eso los mapas UMAP se publican sin números en los ejes.

Saber esto te pone por delante de mucha gente que interpreta estos mapas de más.

Cómo lo usaremos (lo que viene)

1. Instalar `umap-learn` (si Python 3.14 lo permite; hay plan B con PCA/t-SNE de scikit-learn — cambia la pieza, no el concepto).
2. Correr UMAP sobre la matriz $126 \times 768 \rightarrow$ tabla de 126 puntos (x, y).
3. Dibujar el primer scatter: tu primer vistazo al mapa (Apunte 13).
4. Después, agrupar las nubes en subtemas con HDBSCAN (Apunte 14).

En una frase

PCA aplasta buscando el mejor “ángulo de sombra” (rápido pero recto); UMAP reacomoda preservando los vecindarios (ideal para texto). En el mapa resultante: confía en los vecinos y las nubes, desconfía de distancias largas, y los ejes no significan nada.

Apunte 13 — UMAP en acción: el primer mapa de tus 126 papers

Proyecto Froth · aprender Machine Learning construyendo

Qué hicimos

Corrimos UMAP de verdad sobre la matriz 126×768 y salió una tabla 126×2 : una coordenada (x, y) por paper. Las guardamos en `2_Datos/embeddings/umap_2d.npy` y dibujamos el primer scatter (está en `3_Resultados/umap_first_look.png`).

Lo importante del resultado: sin decirle nada sobre minería, ya se ven *estructuras*: una nube grande a la izquierda, un brazo arriba a la derecha, una franja diagonal abajo con un grumo apretado en la esquina. Eso son (probablemente) tus subtemas esperando nombre. El paso siguiente (HDBSCAN) los delimitará formalmente.

Las tres decisiones del código (y por qué)

Concepto: `metric="cosine"` — medir con la misma vara

UMAP necesita decidir quiénes son los “vecinos” de cada punto, y para eso mide distancias. Le dijimos que las mida con **similitud coseno** — exactamente la misma vara de la Fase 2 (Apunte 08). Si la Fase 2 compara papers por coseno y UMAP usara otra medida, el mapa contaría una historia diferente a la del buscador. Consistencia ante todo.

Concepto: `random_state=42` — reproducibilidad

UMAP tiene azar interno (de dónde parte el acomodo). Sin semilla fija, cada corrida daría un mapa *ligeramente* distinto — mismos grupos, otras posiciones. Fijar `random_state=42` hace que cualquiera que corra tu código obtenga **el mismo mapa**. En ciencia (y en tu tesis) eso se llama *reproducibilidad* y es sagrado. (El 42 es una broma-tradición de programadores; cualquier número fijo sirve.)

Concepto: `n_components=2` — cuántas dimensiones de salida

Le pedimos 2 dimensiones porque el objetivo es *dibujar*. Se puede reducir a 5 o 10 (útil a veces para el clustering); lo exploraremos si hace falta.

Ojo / error típico

En el scatter quitamos los números de los ejes **a propósito**. Tras el Apunte 12 ya sabes por qué: en UMAP los ejes no significan nada. Si algún día ves un mapa UMAP con ejes numerados e interpretados (“a mayor X, mayor tal cosa”), desconfía del autor.

Cómo leer tu primer mapa

- **Nubes densas** = grupos de papers que hablan de lo mismo (futuros clusters).

- **Puntos sueltos / puentes** = papers que mezclan temas o que no encajan (candidatos a “ruido” en HDBSCAN).
- **Forma general** (brazo, franja, grumo): sugiere subtemas con distinta cohesión — un grumo apretado es un tema muy específico; una nube amplia, un tema general.

En una frase

UMAP convirtió 768 dimensiones en un mapa 2D reproducible (semilla fija) midiendo vecinos con coseno (la misma vara de siempre), y las nubes aparecieron solas: son los subtemas que HDBSCAN delimitará en el siguiente paso.

Apunte 14 — Clustering: que el algoritmo encuentre los grupos (HDBSCAN)

Proyecto Froth · aprender Machine Learning construyendo

El problema

Tu mapa 2D (Apunte 13) muestra nubes a simple vista, pero “a simple vista” no es un método: necesitamos que un algoritmo delimite los grupos de forma objetiva y repetible. Eso es **clustering no supervisado**: agrupar sin que nadie diga qué es cada grupo ni cuántos hay.

La intuición clásica: K-means (y sus dos trampas)

Concepto: K-means

El método más famoso. Le dices “quiero **K** grupos” y él reparte los puntos en K grupos compactos. Simple y rápido, pero con dos trampas para nuestro caso:

1. **Tienes que adivinar K.** ¿Cuántos subtemas tiene tu topic? Justo eso es lo que NO sabemos (es lo que queremos descubrir).
2. **Obliga a TODOS los puntos a entrar en algún grupo.** El paper de poesía acabaría asignado a un cluster de flotación, contaminándolo.

La práctica: HDBSCAN, clustering por densidad

Concepto: HDBSCAN

En vez de repartir, HDBSCAN busca **zonas densas**: regiones donde hay muchos puntos apretados. Cada zona densa = un cluster. Consecuencias:

- **Decide solo cuántos grupos hay** (nosotros no adivinamos K).
- Lo que no pertenece a ninguna zona densa queda como **ruido** (etiqueta -1), en vez de contaminar un grupo.
- Su parámetro principal es `min_cluster_size`: cuántos puntos mínimo hacen falta para que algo cuente como grupo (en Froth: `MIN_CLUSTER_SIZE = 8`).

Lo que pasó con TUS papers (la historia real)

Con `min_cluster_size=8`, HDBSCAN encontró **3 clusters + 9 papers de ruido** (y con 5, casi idéntico: resultado *estable*, buena señal):

Grupo	Tamaño	Qué resultó ser (Apunte 15)
Cluster 0	49	Geología de pegmatitas y granitos de metales raros
Cluster 2	43	Flotación de lepidolita (el corazón de la tesis)
Cluster 1	25	“Cajón de lo ajeno”: reciclaje/economía del litio + los off-topic
Ruido	9	Puentes generalistas y rarezas

Ojo / error típico

Dos sorpresas que enseñan mucho:

1. **“Ruido” no siempre es basura.** En el ruido cayeron dos clásicos totalmente on-topic (p.ej. *The Effect of Bubble Size on Fine Particle Flotation*, 527 citas). ¿Por qué? Son papers *generalistas* que hablan con varios subtemas a la vez: viven “entre” las nubes, sin pertenecer del todo a ninguna.
2. **Los off-topic no fueron ruido: fueron puestos en cuarentena juntos.** El paper de telecomunicaciones, el de poesía y el de historia cayeron los tres dentro del cluster 1. “No ser del tema” también es algo en común, y forma su propia nube lejos de las nubes mineras.

En una frase

K-means exige adivinar K y fuerza a todos adentro; HDBSCAN agrupa por densidad, decide solo cuántos grupos hay y aparta el ruido. En tus datos: 3 subtemas estables + 9 ruidos, con los off-topic en cuarentena dentro de su propio cluster.

Apunte 15 — Ponerle nombre a las burbujas: c-TF-IDF

Proyecto Froth · aprender Machine Learning construyendo

El problema

HDBSCAN te da grupos llamados “0”, “1”, “2”. Para el mapa necesitamos nombres con significado. ¿Cómo se nombra un grupo de 49 papers *automáticamente*?

Por qué NO sirven las palabras más frecuentes

Si tomas las palabras más repetidas de cada cluster, en TODOS sale lo mismo: “mineral”, “flotation”, “study”, “results”. Frecuente $\neq$ característico. Lo que distingue a un grupo son las palabras que él usa mucho **y los demás poco**.

Concepto: TF-IDF y su versión por clases (c-TF-IDF)

TF-IDF puntúa cada palabra combinando dos factores: qué tan frecuente es en un documento (TF) y qué tan *rara* es en el resto (IDF). Palabra frecuente aquí + rara en los demás = puntuación alta = característica.

La variante **c-TF-IDF** (class-based) aplica esto a clusters: se pegan todos los abstracts de cada cluster como si fueran **un solo mega-documento**, y se comparan los mega-documentos entre sí. Las palabras top de cada mega-documento son la etiqueta del cluster. Es el mismo truco que usa BERTopic por dentro.

Lo que salió con TUS clusters

Grupo	Keywords (top c-TF-IDF)	Lectura
Cluster 0 (49)	melt, crystallization, Nb, Zr, Hf	Geología/petrología de pegmatitas y granitos de metales raros
Cluster 2 (43)	flotation, collector, froth, lepidolite	Flotación de lepidolita: colectores y espumas
Cluster 1 (25)	commodity, old scrap, USGS, women, men	Economía/reciclaje del litio + los papers off-topic (cuarentena)

El mapa tiene sentido mineralógico completo: a un lado la geología del yacimiento, al otro el procesamiento (flotación), y aparte “el resto del mundo”.

Ojo / error típico

Hallazgo de calidad de datos: entre las keywords del cluster de flotación se colaron palabras en *francés* (“du”, “récupération”). Causa: el corpus tiene abstracts en francés (¡Beauvoir está en Francia!) y nuestra lista de *stopwords* (palabras vacías que se ignoran: the, of, and...) solo cubre inglés. No rompe nada, pero ensucia etiquetas. Arreglos posibles: stopwords multilingües o detectar idioma en la limpieza de Fase 1. Lección: los datos reales siempre traen una sorpresa más.

Concepto: el algoritmo propone, el experto valida

Las etiquetas automáticas son un *borrador*: “melt-crystallization-Nb” es útil, pero tú como experto la refinarás a “Petrogénesis de granitos de metales raros”. Esa división de trabajo (máquina propone, humano cura) es exactamente cómo se usan estas herramientas en investigación seria.

En una frase

c-TF-IDF nombra cada cluster con sus palabras características (frecuentes dentro, raras fuera), pegando cada cluster en un mega-documento. Tus 3 subtemas: geología de pegmatitas, flotación de lepidolita, y economía/reciclaje (con los off-topic en cuarentena).

Apunte 16 — El bubble map interactivo (Plotly) y el truco del slider

Proyecto Froth · aprender Machine Learning construyendo

Qué construimos

El entregable estrella de la Fase 3: el **mapa de burbujas interactivo** de tus 126 papers. Existe en dos formas, alimentadas por el mismo código:

1. `3_Resultados/topic_map.html` — archivo autocontenido: doble clic y se abre en cualquier navegador, sin servidor. Compartible.
2. La pestaña **Topic map** de tu web app (`app.py`), con controles en vivo.

Qué se ve: un punto por paper, un color por subtema (geología azul, flotación verde, economía/ajeno naranja, ruido gris), tamaño = citas, la etiqueta de cada subtema flotando sobre su nube, y al pasar el mouse: título, autores, año, citas.

Concepto: Plotly

Librería de gráficos **interactivos**: en vez de una imagen fija (matplotlib), produce HTML+JavaScript con zoom, pan, tooltips y leyenda clicable “gratis”. Por eso el mapa se puede guardar como archivo y abrirse en cualquier lado: el navegador hace el trabajo.

Concepto: una figura, dos caras (arquitectura)

La función `bubble_figure()` en `froth/visualize.py` construye la figura, y DOS clientes la reutilizan: `plot_bubbles()` la guarda como HTML, y `app.py` la muestra en Streamlit. Si mañana mejoras el mapa, mejora en los dos sitios a la vez. Principio general: *el motor no se duplica; se envuelve*.

El truco del slider de granularidad

La web app tiene un slider “min papers per subtopic” que re-agrupa el mapa **al instante**. ¿Cómo puede ser tan rápido, si el pipeline tarda?

Concepto: separar lo lento de lo rápido

El pipeline tiene dos partes muy distintas:

- **Lenta**: UMAP (acomodar 126 puntos desde 768D). Se calcula UNA vez y se cachea.
- **Rápida**: HDBSCAN sobre los puntos 2D ya acomodados (milisegundos) + re-etiquetar.

La clave: la granularidad **no afecta a UMAP**, solo a HDBSCAN. Así que mover el slider solo repite la parte barata. Identificar qué se puede cachear y qué debe recalcularse es una habilidad central de ingeniería de datos — la acabas de usar.

Ojo / error típico

La granularidad es una **decisión de lectura**, no una verdad: con valor bajo verás muchos subtemas pequeños (más detalle, más fragmentación); con valor alto, pocos grandes (más síntesis). No existe “el número correcto de clusters” — existe el que mejor cuenta la historia para tu propósito. Por eso es un slider tuyo y no una constante escondida.

Dónde estamos frente a la competencia

Con este mapa alcanzamos la **paridad** visual/interactiva (hover, zoom, filtros, tamaño por citas) con Connected Papers, Litmaps o VOSviewer. Y ya tenemos diferenciadores que ellos no: mapa por *contenido* (no citas), subtemas auto-nombrados, granularidad en vivo, ruido separable, búsqueda semántica integrada y todo open-source. Los diferenciadores estrella (gaps + borrador de review + módulo tarea/PDFs) llegan en las Fases 4–5. La tabla completa vive en `CLAUDE.md`.

En una frase

Plotly convierte el topic map en un mapa interactivo con dos caras (HTML compartible y pestaña de la app) desde una sola función; el slider de granularidad es instantáneo porque solo repite la parte barata (HDBSCAN 2D), no la cara (UMAP). Paridad con la competencia: lograda; los diferenciadores grandes vienen en F4–F5.

Apunte 17 — El filtro de relevancia: el sistema se limpia solo

Proyecto Froth · aprender Machine Learning construyendo

El problema que TÚ detectaste

Validando el mapa (paso 3.7) viste un cluster etiquetado “*women, men, commodity, masculinities*”. Diagnóstico: los papers infiltrados de la Fase 1 (poesía, telecomunicaciones, historia — colisiones de siglas en las búsquedas) dominaban las keywords de un cluster que también contenía papers legítimos de economía del litio. La causa raíz estaba en el *pull*, no en el mapa.

La solución elegante: medir cada paper contra el TOPIC

Concepto: score de relevancia

Ya tenemos todas las piezas: cada paper tiene su vector, y el **título de tu tesis** también puede tenerlo. Entonces: *relevancia de un paper* = similitud coseno entre su vector y el vector del TOPIC. Un paper de poesía queda lejos del título de una tesis de flotación → score bajo → fuera. El sistema usa **sus propios embeddings para limpiarse a sí mismo** — cero reglas manuales, cero listas negras.

Concepto: el umbral lo dictan los datos, no la intuición

¿Dónde cortar? Miramos la distribución real de los 126 scores:

Infiltrados (poesía, telecom, historia...) 0.495 – 0.543

— **hueco natural** —

Todo lo legítimo (incluida economía del Li) 0.587 – 0.90

Hay un **hueco** entre 0.543 y 0.587: el umbral `RELEVANCE_THRESHOLD = 0.55` cae en medio y separa perfecto. Así se eligen umbrales en serio: mirando dónde los datos ya están separados, no inventando un número bonito.

Dónde vive en el código

- `embed.py` → `relevance_to_topic()`: calcula los scores y los guarda como columna `relevance` en `papers.parquet`.
- `config.py` → `RELEVANCE_THRESHOLD = 0.55` (con el razonamiento comentado).
- `cluster.py` → filtra ANTES de UMAP: la basura ni siquiera participa en el acomodo, así el mapa entero mejora.

Qué cambió en el mapa (iteración real de data science)

- Las etiquetas vergonzosas desaparecieron; el cluster de economía ahora es honesto (*usgs, old scrap* = reciclaje/commodities).
- Emergió un **4º cluster** que estaba enterrado: literatura **en francés** sobre recuperación de Nb-Ta-Sn — probablemente la escuela francesa que estudia Beauvoir. Hallazgo valioso... y lección nueva: *el idioma también agrupa* (SPECTER junta los papers franceses). Refinamiento futuro: ¿traducir abstracts o aceptar ese cluster como subtema válido?

- El ruido subió ($9 \rightarrow 21$): al re-acomodarse el mapa, más papers quedaron entre nubes. Para eso está tu slider de granularidad.

Ojo / error típico

Así se ve el ciclo real del data science: **construir** $\rightarrow$ **mirar** $\rightarrow$ **detectar algo raro** $\rightarrow$ **diagnosticar** $\rightarrow$ **arreglar** $\rightarrow$ **descubrir algo nuevo**. Tu ojo experto detectó el problema; los embeddings dieron la solución; y el arreglo destapó un hallazgo (el cluster francés) que nadie buscaba. Cada iteración enseña.

En una frase

Relevancia = $\text{coseno}(\text{paper}, \text{TOPIC})$; umbral 0.55 elegido por el hueco natural en los datos; filtrado antes de UMAP. Resultado: mapa limpio, etiquetas honestas y un cluster francés sorpresa sobre Beauvoir. El sistema se limpia con sus propias herramientas.

Apunte 18 — Git y GitHub: puntos de guardado y respaldo en la nube

Proyecto Froth · aprender Machine Learning construyendo

Qué es Git

Concepto: Git = máquina de puntos de guardado

Como en un videojuego: cada vez que llegas a un estado que vale la pena conservar, haces un **commit** (“guardar partida”): Git fotografía TODOS los archivos del proyecto en ese instante y guarda la foto para siempre, con fecha, autor y un mensaje. Si mañana rompes algo, puedes volver a cualquier foto anterior. El historial vive en la carpeta oculta `.git`. Lo usa todo programador del planeta, a diario.

Vocabulario mínimo:

- **repo(sitorio)**: el proyecto vigilado por Git.
- `git add .`: poner archivos “en la bandeja” para la próxima foto.
- `git commit -m "mensaje"`: disparar la foto.
- `git status`: ver qué hay en la bandeja / qué cambió.
- **rama (branch)**: una línea de historia. La principal se llama `main`.
- **remote / origin**: el apodo de la copia en la nube.
- `git push`: empujar tus fotos locales a la nube.

Concepto: `.gitignore` = lo que JAMÁS entra en las fotos

Un archivo de texto con la lista de lo prohibido: secretos (`.openalex_key`), la burbuja (`.venv/`), datos regenerables (`2_Datos/`), herramientas descargadas (`tools/`). Así el repo lleva el *código y el conocimiento*, no los pesos muertos ni las llaves. Verificamos con `git ls-files` que ningún secreto entró: cero.

Qué es GitHub (y qué NO es)

GitHub es la nube donde respaldas los repos de Git — y además tu **portafolio público**: cualquiera puede ver tu código, clonarlo (`git clone`) y correrlo en su PC. GitHub *guarda* el código; no lo *ejecuta* por el visitante.

Tu repo: `github.com/kmortizva-data/froth` — de momento **privado** (GitHub muestra 404 a los extraños para no revelar ni que existe). Cuando esté pulido: Settings → Change visibility → Public, y nace el portafolio.

Lo que pasó en tu primera vez (incluye los tropiezos, que enseñan)

1. `git init + git add . + git commit` → primera foto: 65 archivos, cero secretos.

2. **Tropiezo 1:** el push falló con “repository does not seem to exist” — el remoto apuntaba a una URL de ejemplo cuya página nunca se creó en GitHub. Lección: *el repo en la nube hay que crearlo antes de empujar* (o dejar que GitHub Desktop lo cree con su botón **Publish repository**, que hace ambas cosas).
3. **Tropiezo 2:** `git remote remove origin` respondió “No such remote” — significaba “ya estaba borrado”, no un error real. Lección: lee el mensaje; a veces el “error” es solo un aviso de que no había nada que hacer.
4. `git branch -M main`: renombrar `master` → `main` (el estándar moderno). El silencio de la terminal = éxito.
5. **Publish repository** en GitHub Desktop → creó el repo en la nube y empujó. Verificado: `main...origin/main` sin diferencias = sincronizado.

Tu rutina desde ahora (la única que necesitas memorizar)

Al final de cada sesión de trabajo con cambios que valgan:

```
git add . → git status (revisar) → git commit -m "qué hice" → git push
```

(O en GitHub Desktop: revisar cambios → escribir resumen → Commit → Push. Es lo mismo con botones.)

En una frase

Git guarda fotos completas del proyecto (commits) y GitHub las respalda en la nube y las exhibe como portafolio; el `.gitignore` protege secretos y pesos muertos. Tu rutina: `add` → `status` → `commit` → `push`. Primera foto: 65 archivos, cero secretos, ya en la nube.

Apunte 19 — Grafos: nodos, aristas, hubs y puentes

Proyecto Froth · aprender Machine Learning construyendo

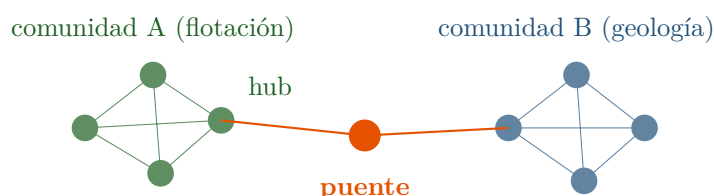
Qué es un grafo

Concepto: grafo = cosas + relaciones

Piensa en tu red social: personas = **nodos**, amistades = **aristas** (las líneas). Esa estructura —cosas conectadas por relaciones— es un **grafo**, y está detrás de Google Maps (cruces + calles), LinkedIn (gente + contactos) y nuestro mapa mental (papers + “se relacionan con”). La librería estándar en Python: **networkx**.

Cuatro ideas que usaremos:

1. **Peso**: no todas las relaciones son igual de fuertes. “Se parecen 0.94” pesa más que “se parecen 0.60” — la línea se dibuja más gruesa.
2. **Dirección**: la similitud es mutua (arista sin flecha); las citas no — A cita a B, no al revés (arista *con* flecha).
3. **Hub**: nodo con muchísimas conexiones.
4. **Puente**: nodo o arista que conecta dos comunidades que casi no se tocan.



Para qué sirve cada uno en TU literature review

Concepto: hubs = cimiento; puentes = originalidad

- **El hub** es la lectura obligatoria: el clásico que todos citan. Va en la introducción de tu review — demuestra que conoces el terreno. Pero *todo el mundo conoce los hubs*: ahí no hay novedad posible.
- **El puente** conecta dos temas que casi no se hablan. Si solo 2 papers unen “flotación” con “geología del yacimiento”, esa frontera está poco explorada → ahí cabe una contribución nueva. Los *gaps* de la Fase 5 son, literalmente, puentes que faltan.

Respuesta corta: hubs para fundamentar, **puentes para aportar**.

Ojo / error típico

Tu tesis ES un puente. Mira el título: “*Flotation of ... Li bearing mica minerals **and its impact on the by-products (Ta, Nb, Be,...) recovery...***”. Conecta exactamente el cluster verde (flotación) con el azul (geología/by-products del granito). No es un tema *dentro* de una comunidad: es la *frontera* entre dos. Por eso es una tesis y no una tarea. Cuando la red esté construida, búscate en ella.

Qué grafo construiremos (lo que viene)

1. **Red de similitud** (4.2): cada paper conectado con sus k vecinos más parecidos por coseno **en 768D** (el espacio real, no el mapa 2D — Apunte 12). Con umbral, para que no salga una “bola de pelo”.
2. **Red de citas** (4.3): flechas de quién cita a quién. Requiere re-bajar los papers pidiendo el campo `referenced_works` que en la Fase 1 no pedimos.
3. **Mapa mental de temas** (4.4): un nodo por subtema, aristas = qué tan relacionados están. La vista presentable.

En una frase

Un grafo son nodos + aristas (con peso y a veces dirección). Los hubs son los clásicos que fundamentan; los puentes, las fronteras donde vive la originalidad — y tu propia tesis es un puente entre flotación y geología. Construiremos redes de similitud, de citas y de temas.

Apunte 20 — La red de similitud: K vecinos, umbral, y tus hubs y puentes reales

Proyecto Froth · aprender Machine Learning construyendo

Cómo se construyó la red

Cada paper se conecta con sus **K papers más parecidos** (similitud coseno calculada en el espacio **original de 768D** — no en el mapa 2D, que ya sabemos que distorsiona), y solo sobreviven las conexiones con similitud $\geq$ un **umbral**. Es el llamado *grafo kNN* (k nearest neighbors).

Concepto: las perillas K y umbral: nadie las “sabe”, se prueban

Preguntaste: ¿y quién dice que 5 vecinos es lo ideal? Respuesta honesta: **nadie** — es una perilla, como la granularidad. La forma seria de elegirla es probar valores y mirar la estructura resultante:

K	líneas	conex./paper	piezas	% entre clusters
3	281	4.7	1	17%
5	457	7.6	1	21%
8	702	11.7	1	24%
15	1213	20.2	1	30%

K alto revela más cruces entre temas (tus primeros vecinos son de tu familia; los lejanos cruzan fronteras) pero con 20 conexiones por paper el dibujo es ilegible (“bola de pelo”). Elegido: **K=5, umbral 0.75** — legible, con puentes visibles y la red en una sola pieza. Ambos viven en `config.py` (`GRAPH_KNN`, `GRAPH_SIM_THRESHOLD`): cambiar de opinión cuesta una línea.

Medir hubs y puentes (ya no a ojo)

Concepto: grado y betweenness

- **Hub** = nodo con muchas conexiones → se mide con el **grado** (contar sus aristas).
- **Puente** = nodo por el que pasan los caminos entre comunidades → se mide con la **betweenness centrality**: cuántas veces un nodo está en el camino más corto entre otros dos. Un paper con *pocas* conexiones puede ser un gran puente si esas pocas unen mundos.

Los resultados con TUS papers

Hubs (las lecturas obligatorias del corpus):

1. *The Magmatic–Hydrothermal Transition in Lithium Pegmatites* (24 conexiones)
2. *Electrostatically Controlled Enrichment of Lepidolite via Flotation* (15)
3. *Mineral Chemistry of Two-Mica Granite Rare Metals* (15)

Puentes (donde vive la novedad) — con dos hallazgos gordos:

1. Entre los 5 puentes top está “*Characterization and Liberation Study of the **Beauvoir Granite***”: el paper fundacional de TU yacimiento es un puente entre comunidades. La red

confirma **con números** que el territorio de tu tesis (geología $\leftrightarrow$ procesamiento en Beauvoir) es terreno-frontera: argumento de originalidad medible para tu introducción.

2. 3 de los 5 puentes top estaban etiquetados como **ruido** por HDBSCAN. La lección del Apunte 14 (“ruido $\neq$ basura; son generalistas entre nubes”) se cumplió cuantitativamente: *el ruido de ayer son los puentes de hoy*.

Dónde vive en el código

`froth/graph.py`: `build_similarity_graph()` (la red, con atributos por nodo), `find_hubs()` (grado), `find_bridges()` (betweenness), `save_graph()/load_graph()` (formato GraphML en `2_Datos/processed/`). Se corre con `python -m froth.graph`.

En una frase

Red kNN: cada paper con sus K=5 más parecidos (coseno en 768D), umbral 0.75 — perillas elegidas probando, guardadas en config. Grado encuentra hubs; betweenness encuentra puentes. Tu corpus confirmó: el paper de Beauvoir es un puente, y el “ruido” escondía a los puentes.

Apunte 21 — El mapa mental de temas (y el hallazgo de los silos)

Proyecto Froth · aprender Machine Learning construyendo

Subir un nivel: de papers a temas

La red del Apunte 20 tiene 120 nodos — buena para explorar, mala para *presentar*. El **mapa mental de temas** agrega la información un nivel arriba: cada **subtema entero se vuelve UN nodo** (tamaño = n° de papers) y las aristas resumen la relación entre subtemas. Es la vista ejecutiva: 3–4 burbujas que cuentan todo el campo.

Concepto: agregación: la misma red, menos resolución

Este truco —agrupar nodos y resumir sus relaciones— se llama *agregación* y es universal: de personas → departamentos, de calles → ciudades, de papers → subtemas. Se pierde detalle, se gana legibilidad. Por eso mantenemos *ambas* redes: la de papers (explorar) y la de temas (entender/presentar).

Las aristas llevan DOS números (y ahí está la gracia)

Entre cada par de subtemas medimos dos cosas distintas:

1. **Similitud** = promedio del coseno entre los papers de uno y otro (en 768D). “¿Qué tan relacionado es lo que *estudian*?”
2. **Citas cruzadas** = cuántas veces un paper de un subtema *cita* a uno del otro. “¿Qué tanto se *leen* entre ellos?”

El hallazgo: silos (tu gap, cuantificado)

Con tus datos salió esto:

Relación	Similitud	Citas cruzadas
Geología ↔ Flotación	0.688	1
Geología ↔ Economía/reciclaje	0.662	3
Flotación ↔ Economía/reciclaje	0.683	1

Ojo / error típico

Lee la primera fila con ojos de tesista: geología y flotación **estudian cosas muy relacionadas** (0.69 es alto) pero **casi no se citan** — ¡1 cita entre 90 papers! De las 116 citas internas del corpus, solo 5 cruzan fronteras de subtema: las comunidades son **silos** — conectadas por significado, separadas por costumbre.

Tu tesis (“flotación de micas de Li *y su impacto en* la recuperación de by-products del granito”) vive exactamente en ese silencio entre silos. Esto es un *gap* cuantificado y citable en tu introducción: la Fase 5 formalizará esta detección.

Dónde vive en el código

`froth/graph.py` → `build_topic_graph()`: nodos con label/nº papers/citas/centroide, aristas con similitud y citas cruzadas. Se imprime al correr `python -m froth.graph`; la vista previa está en `3_Resultados/topic_mindmap.png`.

En una frase

Agregar la red de papers a nivel de temas da la vista ejecutiva (3–4 burbujas). Cada arista lleva similitud (qué estudian) y citas (qué se leen) — y en tu corpus divergen: subtemas muy afines que no se citan = silos = tu gap, ya con números.

Apunte 22 — Las vistas finales: red con física y circle packing

Proyecto Froth · aprender Machine Learning construyendo

Tres vistas, tres preguntas

Tu app ahora tiene **cuatro pestañas**, y cada vista responde una pregunta distinta:

Vista	Responde a...	Posición significa...
Topic map (scatter)	¿Qué grupos hay y qué tan cerca están?	Distancia semántica (UMAP)
Network (red)	¿Quién se conecta con quién? ¿Hubs? ¿Puentes?	La física acomoda; los enlaces son lo real
Packed bubbles	¿Cómo se reparte el campo? (presentar)	Nada — solo pertenencia y tamaño

Saber *qué significa la posición en cada vista* te salva de sobre-interpretar: en el scatter la cercanía ES información; en el packing es solo estética.

La red con física (pyvis)

Concepto: layout force-directed (dirigido por fuerzas)

La pestaña Network simula **física**: cada enlace es un *resorte* que atrae a los papers conectados, y todos los nodos se *repelen* un poco entre sí. El sistema se suelta y busca equilibrio solo: las comunidades muy conectadas quedan juntas y compactas, los puentes quedan estirados en medio. Por eso al **arrastrar un nodo** toda la red reacciona — esa es la “animación”: no es decorado, es el equilibrio recalculándose. La librería: **pyvis** (envuelve vis.js, un motor JavaScript).

El circle packing (circlify)

Concepto: empaquetado de círculos

La vista Packed bubbles coloca un **círculo grande por subtema** y dentro empaqueta un círculo por paper (área = citas), **sin solapes**. Calcular dónde va cada círculo para que quepan apretados es un problema geométrico clásico; la librería **circlify** lo resuelve y nosotros solo dibujamos (con Plotly, para conservar el hover). Es la vista “de presentación” — la del gráfico de packaging de la UE que inspiró el proyecto.

Ojo / error típico

En el packing, dos papers vecinos NO son más parecidos que dos lejanos: la posición dentro del círculo la decide la geometría del empaquetado, no el significado. Membresía (qué círculo grande) y tamaño (citas) son la única información. Elegante para comunicar; no para analizar.

Dónde vive todo

- `froth/visualize.py`: `network_html()` (red pyvis autocontenida), `packed_figure()` (packing con circlify+Plotly), y sus versiones `save_*/plot_*` que escriben HTML en `3_Resultados/`.
- `app.py`: pestañas **Network** y **Packed bubbles** nuevas. La red se cachea por versión de datos (mismo truco del Apunte del slider).
- Archivos compartibles: `topic_map.html`, `paper_network.html`, `packed_bubbles.html` — doble clic y se abren en cualquier navegador.

En una frase

Tres vistas para tres preguntas: scatter (distancias semánticas), red con física (conexiones, hubs, puentes — arrastrable porque es un sistema de resortes) y circle packing (presentación pura, posición sin significado). Todas en la app y como HTMLs compartibles.

Apunte 23 — DOI, open access y la lista de lectura

Proyecto Froth · aprender Machine Learning construyendo

La feature nueva: del ranking a la lectura

El buscador ya rankeaba papers; ahora además te lleva a **leerlos**: cada resultado trae dos enlaces, **DOI** (la página oficial del paper) y **open PDF** (el PDF gratuito, cuando existe). En tu corpus: **85 de 126 papers (67%) tienen PDF de acceso abierto** y 116/126 tienen DOI.

Concepto: DOI (Digital Object Identifier)

El **DOI** es el “número de cédula” permanente de un paper: 10.1016/j.... La URL [https://doi.org/\[DOI\]](https://doi.org/[DOI]) siempre lleva a la página oficial del editor, aunque la revista cambie de sitio web. Es la forma correcta de citar y de llegar al texto; si el paper es de pago, ahí usas tus accesos de universidad.

Concepto: open access (OA)

Un paper es **open access** cuando existe una copia legalmente gratuita: en la propia revista, en repositorios (arXiv, HAL...) o en la página de la universidad del autor. OpenAlex rastrea dónde está esa copia y nos da la URL (`oa_url`). Nosotros solo pedimos un campo más en el **select** del pull (`open_access`) — una línea de código, y el pipeline regeneró todo lo demás solo.

Ojo / error típico

Enlazar sí, descargar masivamente no. Froth te da el ENLACE y tú haces clic: la lectura ocurre de tu lado, con tus permisos. Descargar PDFs en masa desde editoriales viola sus términos de uso (riesgo previsto en el plan desde el día 1). Las herramientas serias (Connected Papers, etc.) hacen exactamente esto mismo: rankear + enlazar.

Bonus del día: el vigilante lento de Streamlit

La app tardaba mucho en arrancar y el log se llenaba de avisos de `torchvision`. Diagnóstico: el *file watcher* de Streamlit (auto-recarga al editar código) intenta examinar los cientos de módulos de `transformers`, uno por uno. Solución: apagarlo en `.streamlit/config.toml` (`fileWatcherType = "none"`). Lección doble: (1) un log ruidoso no siempre es un error fatal — hay que leer QUÉ dice; (2) las herramientas de desarrollo cómodas a veces cuestan rendimiento.

En una frase

Cada resultado del buscador ahora enlaza al DOI (página oficial) y al PDF open access cuando existe (67% de tu corpus). Enlazamos, no descargamos en masa. Y la app arranca más rápido tras apagar el file watcher de Streamlit.

Apunte 24 — Qué es un gap (y cómo se mide de verdad)

Proyecto Froth · aprender Machine Learning construyendo

Por qué importan

Toda tesis debe aportar algo nuevo, y “nuevo” casi siempre significa **trabajar donde otros no han trabajado**. El problema: la frase “research gap” se usa mucho y se mide poco — suele ser una corazonada del supervisor. El diferenciador de Froth es convertir el gap en una **señal medible sobre los datos**, con evidencia que puedas citar.

Los 4 tipos de gap medibles

Concepto: 1. El SILO (el más valioso)

Dos comunidades que estudian cosas **muy parecidas** pero **no se citan**: trabajan de espaldas. Se mide restando dos matrices que ya tenemos: similitud semántica entre clusters (alta) vs. citas cruzadas (bajas). **Tu corpus lo tiene**: geología ↔ flotación = similitud 0.69 con **una sola cita cruzada** (Apunte 21). Dos mundos hablando del mismo granito sin hablarse. Quien conecte esos mundos, aporta.

Concepto: 2. La ZONA RALA

Regiones vacías del mapa *entre* nubes densas: combinaciones de temas que nadie tocó. Se mide con densidad espacial en el espacio de embeddings (¿cuántos papers viven entre el centroide de A y el de B?).

Concepto: 3. El PUENTE ESCASO

Entre dos clusters solo existen 1–2 papers puente (la betweenness del Apunte 20 concentrada en poquitos nodos). Si cruzar ese puente produce valor y casi nadie lo ha cruzado, hay sitio para más viajeros.

Concepto: 4. El TEMA DORMIDO

Un subtema cuyos papers son todos viejos: conocimiento que nadie revisita con métodos modernos. Se mide con la distribución de años por cluster (¿cuándo fue el último paper?).

La advertencia: mina o desierto

Ojo / error típico

Un hueco puede estar vacío por dos razones: porque **nadie lo exploró** (una mina esperando) o porque **no hay nada que encontrar** (un desierto — a veces la combinación de temas es estéril por buenas razones físicas o económicas). **Ningún algoritmo distingue una de otra.** Por eso Froth *propone candidatos con evidencia* y el experto decide. Desconfía de cualquier herramienta que prometa “gaps automáticos garantizados”; la nuestra promete candidatos honestos con sus números.

Tu tesis ya es la prueba del concepto

El título de la tesis (flotación de micas de Li **y su impacto en** la recuperación de by-products del granito de Beauvoir) explota *exactamente* el silo #1: une el cluster de flotación con el de geología. Fareed encontró ese gap a ojo y con experiencia; Froth lo encontró con números — similitud alta, una cita cruzada, y el paper de caracterización de Beauvoir como puente solitario (Apunte 20). Esa doble confirmación (experto + datos) es la validación más fuerte que puede tener el método.

Lo que viene (5.1)

Implementar las cuatro señales en `gaps.py` → `find_gaps()` devolverá una tabla rankeada: cada gap candidato con su tipo, su evidencia (similitud, citas, densidad, años) y los papers frontera que ya lo rondan. Después (5.2), el borrador de review citará esos gaps con números — no con corazonadas.

En una frase

Un gap medible tiene 4 formas: silos (parecidos que no se citan), zonas ralas, puentes escasos y temas dormidos. La herramienta propone candidatos con evidencia; el experto distingue minas de desiertos. Tu propia tesis es el silo geología-flotación de Beauvoir, ahora confirmado con números.

Apunte 25 — De la teoría al texto: `find_gaps()` y el borrador de review

Proyecto Froth · aprender Machine Learning construyendo

Qué construimos (5.1 + 5.2)

Las dos mitades del diferenciador, ya funcionando en `froth/gaps.py`:

1. `find_gaps()`: convierte los 4 tipos del Apunte 24 en señales medidas → tabla rankeada con evidencia (`2_Datos/processed/gaps.parquet`).
2. `draft_review()`: redacta el andamio del literature review (`3_Resultados/review_draft.md`) con **citas 100% trazables**.

Se corren juntas: `python -m froth.gaps`.

Los gaps de TU corpus (resultados reales)

Tipo	Candidato top	Evidencia
Silo	commodities/críticos ↔ flotación	sim 0.69, 1 cita cruzada
Silo	flotación ↔ geología (<i>tu tesis</i>)	sim 0.70, 3 citas cruzadas
Puente escaso	commodities ↔ geología	solo 8 puentes pese a sim 0.67
Dormido	(ninguno: campo caliente, papers 2025-26)	la herramienta lo dice honesto

Lectura fina del silo campeón: la literatura de *criticidad/cadenas de suministro* y la de *procesamiento* casi no se citan. Conectarlas (“mi flotación importa para la seguridad de suministro de Li/Ta”) es munición directa para introducciones y propuestas.

Ojo / error típico

La señal más débil es la **zona rala**: mide *ausencias*, y un vacío no explica su porqué (¿mina o desierto?) — además de heredar las distorsiones del mapa 2D (Apunte 12). Por eso va etiquetada “experimental” en el código y en el draft. Regla general: desconfía más de las métricas sobre lo que falta que de las métricas sobre lo que existe.

El borrador: por qué una plantilla y no un LLM

Concepto: citas trazables o nada

El draft se genera con una **plantilla determinista**: cada sección nace de la estructura MEDIDA (secciones = clusters; imprescindibles = más citados + hubs locales; novedades = más recientes; huecos = gaps con sus números) y cada afirmación apunta a papers reales con referencia numerada [n] → lista final con autor/año/título/DOI. Un LLM redactaría más bonito... y tarde o temprano *inventaría* una referencia — el pecado capital académico. Orden correcto: primero el andamio confiable, después (opcional) un LLM que pula la prosa *verificando* cada cita contra el corpus.

Frente a la competencia (otro vecindario)

Aquí los rivales no son los mapeadores sino Elicit/SciSpace/Consensus (LLM que sintetiza *sin* conocer la estructura del corpus), Scholarcy (resúmenes 1-a-1) y ChatGPT a pelo (referencias alucinadas). **Nadie une mapa y texto:** nuestro draft dice “estos dos subtemas comparten 0.69 de similitud y una sola cita cruzada” porque lo midió. Límite honesto: trabajamos con abstracts (Elicit extrae del full text) → lo nuestro es un *andamio estructurado*, no prosa final.

En una frase

`find_gaps()` produce candidatos rankeados con evidencia (silos al frente: commodities-flotación 0.69/1 cita; tu tesis reconfirmada 0.70/3); `draft_review()` los convierte en un andamio de review con 26 referencias trazables, sin LLM. Nadie más une mapa y texto.

Apunte 26 — El modo Tarea por dentro: trocear y promediar (mean pooling)

Proyecto Froth · aprender Machine Learning construyendo

El problema que no se ve

El modo Tarea acepta PDFs de instrucciones enteros. Pero SPECTER (como casi todos los modelos de embeddings) solo “lee” unos **512 tokens** ($\approx 350\text{--}400$ palabras): todo lo que pase de ahí lo **trunca en silencio**. Si le dieras un PDF de 10 páginas, el vector representaría... la primera página. El resto, ignorado sin aviso. Este tipo de límite silencioso es una trampa clásica de los modelos de lenguaje.

La solución: trocear y promediar

Concepto: chunking + mean pooling

`embed.embed_texts_mean()` hace dos pasos:

1. **Trocear (chunking)**: parte cada texto en pedazos de ~ 180 palabras (cómodamente bajo el límite).
2. **Promediar (mean pooling)**: vectoriza cada pedazo y calcula el **promedio** de todos los vectores $\rightarrow$ un solo vector que representa el documento COMPLETO.

La intuición geométrica: cada pedazo es un punto en el mapa del significado; el promedio es el *centro de gravedad* de todos — la posición media de lo que el documento trata.

Ojo / error típico

El promedio también *diluye*: si el PDF mezcla temas muy distintos, el centro de gravedad cae “entre” ellos y puede no representar bien a ninguno. Para instrucciones de tareas (temáticamente coherentes) funciona muy bien; para documentos-miscelánea habría que agrupar pedazos por tema primero. Limitación conocida y aceptada.

La prueba real

Consigna simulada: “*informe técnico comparando colectores para flotación de lepidolita y su selectividad frente a ganga de feldespato y cuarzo*” (repetida hasta forzar varios pedazos). Resultado: top-5 puros papers de colectores, encabezados por el del colector LCJ (similitud 0.897). El centro de gravedad cayó exactamente donde debía.

En una frase

Los modelos de embeddings truncan a ~ 512 tokens en silencio; el modo Tarea lo esquivo troceando el texto y promediando los vectores (mean pooling = centro de gravedad del significado). Probado: una consigna de colectores devuelve papers de colectores.

Apunte 27 — Granularidad recomendada: que los datos elijan (DBCV)

Proyecto Froth · aprender Machine Learning construyendo

La pregunta de Batman que originó esto

“Quiero un botón de *ajustar a recomendado* con una razón estadística REAL — no un valor estándar, sino calculado cada vez.” Exactamente la actitud correcta: los defaults fijos son opiniones; los datos cambian con cada corpus.

La métrica correcta: DBCV

Concepto: DBCV (Density-Based Cluster Validity)

Para evaluar un clustering *por densidad* (HDBSCAN) no sirve cualquier métrica: la famosa *silhouette*, por ejemplo, asume clusters redondos y compactos. **DBCV** es la métrica nativa de la familia: premia clusterings cuyos grupos son **densos por dentro** y están **bien separados por fuera**, respetando formas irregulares. En código: `hdbscan.HDBSCAN(gen_min_span_tree=True).relative_validity_`.

Concepto: el barrido (sweep)

`cluster.recommend_min_cluster_size()` prueba TODAS las granularidades de 3 a 20, calcula la validez DBCV de cada una, descarta las degeneradas (1 solo cluster no es un clustering) y devuelve la ganadora + una tabla de diagnóstico. Con el corpus ampliado de hoy: recomendado = 7 (validez 0.51). Si mañana el corpus cambia, se recalcula solo (cacheado por versión de datos).

Ojo / error típico

La recomendación es un *óptimo estadístico*, no una verdad final: optimiza densidad y separación, no “qué historia le sirve al experto”. Por eso el slider sigue ahí — divagar está permitido y el botón siempre te trae de vuelta. Máquina propone; humano dispone (el patrón de todo Froth).

En una frase

El botón “Use recommended” no guarda un número mágico: barre granularidades y elige la que maximiza DBCV — la métrica de calidad nativa del clustering por densidad — recalculada para cada corpus. Estadística real, con el slider libre para explorar.

Apunte 28 — Conectores multi-fuente: tu cuenta institucional entra al juego

Proyecto Froth · aprender Machine Learning construyendo

Por qué no basta OpenAlex

OpenAlex es enorme (~250M trabajos) pero no lo tiene todo, y a veces le faltan abstracts. La idea de Batman: que el usuario **conecte sus propias cuentas** (estilo “Connect with Google”) y vea papers que la base gratuita no trae.

Los conectores (froth/sources.py)

Fuente	Requiere	Qué aporta
OpenAlex	nada (key opcional)	la BASE: abstracts + referencias
Crossref	solo un email	metadatos masivos de editoriales
Semantic Scholar	key GRATIS (opcional)	abstracts + PDFs open access
Scopus (Elsevier)	key institucional	el catálogo curado de Elsevier
ResearchGate	NO : sin API pública; su ToS prohíbe scraping	

Principios de diseño:

- **OpenAlex manda:** es el registro preferido (el más rico); los extras solo AÑADEN papers que no teníamos.
- **Dedup por DOI y luego por título** normalizado — el mismo paper llega por varias fuentes y debe contar una vez.
- **Fallo suave:** si una fuente está caída o te limita, avisa y el pull sigue con las demás. Nunca un conector tumba la cosecha.
- **Las keys son tuyas y locales:** viven en `.froth_keys` (gitignored). En la web pública vivirán solo en tu sesión de navegador — Froth jamás custodia keys ajenas en un servidor.

La primera cosecha multi-fuente (números reales)

- Crossref: 227 resultados crudos → el dedup reconoció **58 duplicados** de OpenAlex y aportó **169 nuevos** (tras el filtro de abstract: corpus 126 → **157**).
- Semantic Scholar sin key: rate-limit inmediato → **falló suave** tal como se diseñó (con la key gratuita queda estable).
- El corpus ampliado reveló un **subtema emergente**: física de burbujas/partículas finas (+ bastnäsita) — antes enterrado, ahora cluster propio. Más datos → más resolución del mapa.
- El filtro de relevancia solo descartó 6: la cosecha extra venía limpia — el auto-lavado del Apunte 17 escaló sin tocarlo.

En una frase

sources.py suma Crossref/S2/Scopus a OpenAlex con dedup DOI→título y fallo suave; las keys son locales y del usuario (panel “Connect institutional account”). Primera cosecha: +31 papers netos y un subtema nuevo emergió. ResearchGate queda fuera por diseño (sin API legal).

Apunte 29 — Fase 6: generalizar no es entrenar (y el motor multi-topic)

Proyecto Froth · aprender Machine Learning construyendo

La aclaración que abre la fase

El plan de Batman: “iterar cada uno de los 22 títulos del cohorte y seguir entrenando”. Correcto en espíritu — pero son **dos actos distintos**:

Concepto: generalizar (6.1–6.3) vs. entrenar (6.5)

- **Iterar los 22 títulos NO entrena nada:** el modelo (SPECTER) no cambia ni un peso. Lo que se endurece es el **código** — cada título raro que rompe algo obliga a arreglarlo. Es la escalera 1→2→22 del plan original.
- **Entrenar de verdad (6.5)** sí cambia los pesos del modelo: fine-tuning de SPECTER con los miles de abstracts acumulados de los 22 corpus. PyTorch, loss, épocas — en la GPU de Batman (RTX 5060, confirmada capaz).

Los dos actos se alimentan: la generalización produce el dataset del entrenamiento.

El motor multi-topic (6.1)

Hasta ayer el topic vivía *clavado* en `config.py`. Ahora:

Concepto: `config.set_topic(título)`

Una llamada re-apunta TODO el motor: deriva términos de búsqueda nuevos y mueve las rutas de datos a una carpeta propia (`2_Datos/topics/<slug>/`). Funciona porque cada módulo lee `config.*` *en el momento de ejecutar*, no al importar — un detalle de diseño que pagó dividendos. (Trampa clásica cazada: `def f(x=config.TOPIC)` congela el valor al importar; se corrigió a `x=None` + resolver adentro.)

Concepto: `derive_search_terms()`: de título a búsquedas

Los términos del topic 1 se escribieron a mano. Para CUALQUIER título, la heurística de `froth/terms.py`: quita palabras de relleno (“innovative approaches for the..”), conserva **frases reales del título** (bigramas/trigramas consecutivos: *froth flotation*, *scheelite ores*, *Quantitative XRD*), respeta **acrónimos** (XRD, LIBS, NiMH) y extrae el **contenido de paréntesis** (ahí viven los elementos y yacimientos). Determinista, sin dependencias. La capa LLM local (Ollama) vendrá encima como refinador opcional — decisión ya tomada: local, privado, gratis.

El hito 1→2 (probado)

Pipeline completo sobre “*scheelite ores by froth flotation*” **sin tocar código**: 47 papers, carpeta propia, y clusters con sentido metalúrgico (concentrador Falcon, reactivos SNF, espumantes). Y la prueba pagó de inmediato:

Ojo / error típico

Lo que el 2º topic enseñó (por esto existe la escalera):

1. La keyword “nbsp” delató que los abstracts de Crossref traen **entidades HTML** () — el limpiador ahora hace `html.unescape`.
2. El `.gitignore` no cubría las carpetas nuevas por topic — los datos casi acaban en GitHub.
3. En corpus chicos (47 papers) la granularidad fija de 8 deja demasiado ruido → el pipeline ahora acepta `min_cluster_size="auto"` (elige por DBCV, Apunte 27, por corpus).

Tres arreglos reales con UN solo título nuevo. Los 22 del batch encontrarán más — ese es exactamente su trabajo.

El desfile (6.2)

`froth/batch.py` lee los 22 títulos del CSV del cohorte y corre el pipeline para cada uno, con reporte de robustez por topic (papers, relevantes, clusters, ruido, granularidad elegida, segundos, estado). Diseño defensivo: el reporte se guarda tras **cada** topic y las corridas siguientes **reanudan** donde quedaron (un rate-limit no pierde nada). Los fallos no detienen el desfile: quedan como FAILED en la tabla — y esa tabla es la agenda de arreglos de la 6.3.

En una frase

Iterar títulos endurece el código (generalizar); cambiar pesos es entrenar (6.5). El motor ya es multi-topic (`set_topic` + términos derivados + carpetas por topic), probado con scheelite ores — que de paso regaló 3 arreglos. El batch de 22 corre con reanudación y reporte por topic: su tabla de fallos es el plan de trabajo de la 6.3.

Apunte 30 — El nacimiento de specter-mineral: tu primer entrenamiento real

Proyecto Froth · aprender Machine Learning construyendo

Qué pasó (en 83 segundos)

Tu RTX 5060 tomó el SPECTER genérico y lo **afinó** con los $\sim 1,250$ pares título $\leftrightarrow$ abstract del corpus maestro: 2 épocas, 160 pasos, lotes de 16. El resultado vive en `models/specter-mineral` — un modelo de embeddings que habla el dialecto del procesamiento mineral. En CPU habrían sido horas; en tu GPU, 83 segundos.

Los conceptos que acabas de usar

Concepto: fine-tuning (afinar) $\neq$ entrenar desde cero

No construimos un modelo nuevo: tomamos los pesos de SPECTER (que ya sabe “leer papers”) y los **empujamos suavemente** hacia tu dominio. Como un geólogo generalista haciendo una especialización — no vuelve a la primaria.

Concepto: la jugada elegante: auto-supervisión con MNRL

Entrenar suele exigir etiquetas humanas (“estos dos textos son parecidos”). No etiquetamos NADA. El truco: **MultipleNegativesRankingLoss** — en cada lote de 16 papers, el título de un paper debe quedar más cerca de *su propio* abstract que de los otros 15 del lote. Positivos gratis (título y abstract del mismo paper hablan de lo mismo) y negativos gratis (los demás del lote). Un corpus plano se vuelve dataset de entrenamiento sin mover un dedo.

Concepto: el vocabulario del entrenamiento

- **loss (pérdida)**: el número que mide “qué tan equivocado va” el modelo; entrenar = empujar los pesos para bajarlo.
- **paso (step)**: procesar un lote y ajustar pesos una vez. Viste “160 pasos”.
- **época (epoch)**: una pasada completa por todo el dataset. Hicimos 2.
- **warmup**: los primeros pasos con ajustes suavecitos, para no desbaratar lo que el modelo ya sabía.
- **mixed precision (amp)**: cuentas en números de menor precisión donde no importa — el motivo de parte de la velocidad en GPU.

Ojo / error típico

Los tropiezos también enseñan: el entrenamiento falló DOS veces antes de arrancar — faltaban `datasets` y luego `accelerate` (el kit del Trainer de Hugging Face). Lección de ecosistema: en ML moderno “instalar la librería” rara vez basta; el stack de entrenamiento es su propia bestia. Ambas quedaron en `requirements.txt` para siempre. Y antes de eso: pip se negó a cambiar torch CPU $\rightarrow$ CUDA porque “la versión ya estaba satisfecha” (mismo número, distinta variante) — se resolvió con `--force-reinstall --no-deps`.

Cómo se juzga si valió la pena (6.6)

Un modelo nuevo no es mejor por existir. `froth/compare.py` somete a AMBOS modelos al mismo examen sobre los 22 topics: mismos corpus, mismo UMAP (semilla fija), granularidad elegida por DBCV — ¿quién produce clusters más válidos? Además exporta un **test ciego**: dos listados de clusters A/B con la clave escondida, para que el experto (tú) juzgue sin saber quién es quién. Si specter-mineral no gana, eso TAMBIÉN es un resultado honesto y publicable.

En una frase

Fine-tuning = empujar pesos existentes hacia tu dominio; MNRL convierte el corpus en dataset sin etiquetar (tu título debe amar a tu abstract más que a los ajenos); 83s en tu GPU tras dos tropiezos de dependencias. El veredicto lo dan DBCV en 22 topics y tu test ciego — no el entusiasmo.

Apunte 31 — El veredicto del fine-tuning: especialización, no magia

Proyecto Froth · aprender Machine Learning construyendo

El experimento (6.6)

Dos modelos compitieron sobre los 22 corpus del cohorte con el MISMO examen (mismos datos, misma semilla de UMAP, granularidad elegida por DBCV): el SPECTER genérico contra **specter-mineral**, tu modelo afinado en 83 segundos de GPU con ~ 1250 pares título $\leftrightarrow$ abstract.

Los números (la historia está en la varianza)

- **specter-mineral ganó 14/22 topics** en validez DBCV.
- Pero el delta *medio* fue $+0.007$ con $\sigma = 0.170$ — un promedio plano escondiendo un patrón fuerte:
- **Ganó grande en el núcleo del dominio:** reciclaje hidrometalúrgico ($+0.30$), escorias XCT ($+0.24$), core imaging de Beauvoir ($+0.20$), tu tesis ($+0.05$).
- **Perdió en la periferia:** morteros geopolímeros (-0.43), simulación de refinería (-0.27), IA en cintas (-0.21).

Concepto: la lección: el fine-tune pequeño especializa hacia el centro de masa

Con pocos datos, afinar no “mejora el modelo en general”: lo **arrastra hacia donde está la masa del corpus de entrenamiento** (flotación/geología). Los topics cercanos a esa masa ganan resolución; los lejanos la pierden. Hipótesis abierta (tu próximo experimento natural): ¿el corpus $2\times$ con Scopus reduce la penalización periférica?

El doble ciego (tu momento)

Dos listados de clusters del corpus de Beauvoir, etiquetados A/B al azar, clave oculta. Elegiste **B** “porque separaba la flotación de finos de la química de colectores de lepidolita” — y B era **tu modelo**. Métrica y experto coincidieron en el dominio que dominas: la validación más fuerte disponible.

Ojo / error típico

Un “no mejora en promedio” con estructura clara (gana aquí, pierde allá, y sabemos por qué) es MEJOR resultado científico que un “ $+2\%$ en todo” inexplicado. No infles: matiza.

En una frase

14/22 victorias, delta medio ≈ 0 , σ grande: el fine-tuning pequeño especializa hacia el núcleo del corpus a costa de la periferia — confirmado por métrica (DBCV) y por test doble ciego donde elegiste tu propio modelo sin saberlo.

Apunte 32 — h-index por cluster: cuántos papers son “imperdibles”

Proyecto Froth · aprender Machine Learning construyendo

El problema (tu propia orden anti-números-mágicos)

En la guía de lectura queríamos marcar los must-reads de cada subtópico. ¿Top 3? No: “top 3” es un número mágico — igual de arbitrario para un cluster de 110 papers de geología que para uno de 9 de tomografía. Pediste *investigar cómo investigar*: un umbral calculado **por corpus y por cluster**.

La idea robada de la bibliometría: el h-index

El h-index se inventó para medir investigadores, pero la definición funciona sobre CUALQUIER lista de conteos de citas:

Concepto: h-index de un cluster

El mayor h tal que el cluster tiene h papers con al menos h citas cada uno. Ordenas por citas descendentes y buscas el punto donde el rango supera a las citas: justo ahí la evidencia deja de sostenerse a sí misma.

Por qué es elegante: **el propio cluster fija su vara**. Un cluster mega-citado exige cientos de citas para entrar al must-read; uno joven se conforma con decenas. Cero parámetros que tunear.

Los datos reales de Beauvoir

- Geología/granitos (110 papers) → $h = 35$: literatura madura, la vara quedó altísima sola.
- Tomografía XCT (9 papers) → $h = 3$: nicho joven, vara proporcional.
- Informes USGS (mayoría con 0–2 citas) → el h-index degenera ($h > n/2$, marcaría “casi todo es imperdible”): ahí entra el **fallback head/tail breaks** — se corta en la media de citas del cluster, que en distribuciones de cola larga separa la cabeza pesada del resto (→ 7).

Ojo / error típico

Citas ≠ calidad: miden atención acumulada, castigan lo reciente (los 59 papers de 2025–2027 con 0 citas son investigación fresca, no mala). El must-read dice “esto es lo canónico que el campo espera que hayas leído”, no “esto es lo único que vale”.

Dónde vive esta regla (y una lección de 2026-08-03)

Hasta hoy este apunte describía un fallback que **nunca se había programado**: el h-index sí estaba en el código, el corte por la media no. Se implementó al construir el export a Obsidian, en `froth/export_obsidian.py` (`_must_reads` y `_mean_cut`).

Ojo / error típico

Un desvío pequeño rompió la regla entera. Al programarlo, Claude repitió el corte por la media *hasta seis veces* en vez de hacerlo UNA, creyendo que así sería más selectivo. Efecto medido sobre tu corpus de tesis: en el subtema de 340 papers hay un clásico con 4.250 citas que arrastra la media, y al repetir el corte la lista se estrangulaba hasta **1 paper**. Con un solo corte da 36. En total, la versión repetida dejaba con 2 papers o menos a **20 de los 29 subtemas de 30+ papers**; la regla de este apunte no lo hace en ninguno. Lección: cuando un apunte fija un método, el código debe seguirlo *literalmente*. “Lo hago un poco más estricto” no es una mejora, es otro método, y hay que medirlo antes de llamarlo mejor.

Números actuales con la regla correcta: corpus de tesis, 3.867 papers en 104 subtemas → 893 must-reads (mediana 6 por subtema, ninguno se queda corto).

En una frase

Must-reads sin números mágicos: h-index por cluster (auto-calibrado a la madurez de cada subtópico) con fallback head/tail breaks cuando las citas son demasiado planas para que el h-index discrimine.

Apunte 33 — Resortes calibrados y papers frontera: qué significa la posición en la red

Proyecto Froth · aprender Machine Learning construyendo

Tu pregunta: “¿los resortes significan algo?”

En la vista Network los nodos se acomodan con un **force layout** (forceAtlas2): cada arista es un resorte que junta, y todos los nodos se repelen como cargas. El equilibrio de ese tira-y-afloja decide la posición. Por defecto vis.js usa la MISMA longitud de resorte para toda arista — la distancia visual era pura estética.

La calibración: longitud = disimilitud

Nuestras aristas ya traen un peso w = similitud coseno (con umbral 0.75). Ahora cada resorte se fabrica a medida:

$$\text{length} = 60 + 420(1 - w)$$

Vecinos casi idénticos ($w \rightarrow 1$) quedan a ~ 60 px; los que apenas pasaron el umbral, a ~ 165 px. Desde ese cambio, **cerca en pantalla** $\approx$ **cerca en significado** — la red responde la misma pregunta que el mapa UMAP, pero con las conexiones explícitas.

El misterio del “amarillo pegado al hub azul”

Notaste nodos de un color metidos en el territorio de otro. No era bug: eran **papers frontera**. La cadena de eventos:

1. HDBSCAN corta los clusters en el mapa **2D** de UMAP.
2. La red conecta vecinos en los **768D** originales.
3. Un paper puede caer del lado amarillo del corte 2D pero tener a casi todos sus vecinos semánticos en el cluster azul $\rightarrow$ los resortes lo arrastran hacia el azul.

Concepto: detección de papers frontera

Para cada nodo se cuenta el cluster de sus vecinos. Si $\geq 60\%$ pertenece a OTRO cluster, es frontera: se dibuja con **anillo discontinuo del color del cluster vecino** y el tooltip lo declara (“bridges into the neighboring subtopic”). En Beauvoir: 3 de 237 papers.

Ojo / error típico

Los papers frontera no son errores de clustering: suelen ser los interdisciplinarios — exactamente los que conectan subtópicos. Para un lector buscando huecos de investigación, son de los más interesantes del mapa.

En una frase

Resortes con $\text{length} = 60 + 420(1 - w)$: la proximidad visual ahora ES distancia semántica. Los nodos “mal ubicados” son papers frontera (mayoría de vecinos en otro cluster), ahora marcados con anillo discontinuo del color vecino.

Apunte 34 — Resumen extractivo vs abstractivo: por qué el nuestro no puede alucinar

Proyecto Froth · aprender Machine Learning construyendo

El problema de la Fase 7

El review v1 es un andamio honesto pero seco: keywords + listas de papers. Para que *diga* algo hay dos caminos, y la diferencia entre ellos es LA decisión de diseño más importante de esta fase.

Concepto: extractivo vs abstractivo

Abstractivo: un modelo GENERA texto nuevo que “suena a” resumen (ChatGPT, Elicit). Fluido, pero el texto no existe en ninguna fuente: puede inventar afirmaciones y — peor — referencias.

Extractivo: el sistema SELECCIONA frases reales que ya existen en los abstracts. Menos fluido, pero cada frase tiene un paper de origen: la cita es trazable *por construcción*. Alucinar es imposible: no se genera ni una palabra.

Por qué elegimos extractivo primero

- **Confianza:** un review académico con UNA referencia inventada muere. ChatGPT “a pelo” es famoso por citar papers que no existen.
- **Trazabilidad gratis:** como la frase viene de un abstract del corpus, el botón “know more” y la cita [n] ya funcionan sin verificación extra.
- **Es ML de verdad:** rankear frases usa embeddings, coseno, grafos y PageRank — todo lo que ya construiste, a una escala nueva (frases, no papers).
- El pulido con LLM (parafrasear frases YA elegidas, verificando citas) queda como capa OPCIONAL encima — nunca como base (7.6).

La cocina: de abstract a frases utilizables

En `froth/review.py`, `split_sentences()` corta cada abstract donde hay puntuación final seguida de mayúscula, y `_self_contained()` descarta lo que no sobrevive solo: fragmentos (<8 palabras), maratones (>60) y frases que arrancan con **anáforas** — “This method...”, “It also...” — porque apuntan a un texto que el lector del review no verá.

Ojo / error típico

El extractivo hereda las limitaciones de su materia prima: trabajamos con ABSTRACTS (no full text), y un abstract mal escrito da frases mediocres. El review v2 sigue siendo un andamio mejorado, no prosa final — honestidad ante todo.

En una frase

Extractivo = seleccionar frases reales (trazable, sin alucinación); abstractivo = generar texto (fluido, riesgoso). Froth construye sobre extractivo y deja el LLM local como pulidor opcional de frases ya elegidas.

Apunte 35 — La frase centroide: el filtro de relevancia al revés

Proyecto Froth · aprender Machine Learning construyendo

La idea (ya la conocías sin saberlo)

En la Fase 3, el filtro de relevancia medía el coseno de cada paper contra el vector del TOPIC y *descartaba* lo lejano. La frase centroide usa la MISMA matemática con la intención opuesta: medir el coseno de cada frase contra el centro del cluster y *coronar* lo más cercano.

Concepto: centroide y frase representativa

El **centroide** de un cluster es el promedio de los vectores (768D) de sus papers: su “centro de masa semántico”. Las frases de los abstracts se vectorizan con el MISMO modelo (SPECTER), así que viven en el mismo espacio y son comparables. La frase con mayor coseno al centroide es la que mejor resume de qué va el subtema — elegida por geometría, no por opinión.

Los resultados reales (corpus Beauvoir, 237 papers)

`python -m froth.review` sobre tus datos:

- Cluster geología (“ma, melt, compositions”): score **0.947** para “*LCT-type granitic pegmatites represent a major global source of lithium...*” — una definición de libro del subtema.
- Cluster USGS (“usgs, old scrap, year”): “*Minerals Yearbook — These annual publications review the mineral industries of the United States...*” Las keywords jamás te dijeron que ese cluster son INFORMES ANUALES; la frase sí.
- Cluster flotación de lepidolita: “*Efficient flotation of lepidolite is crucial for the extraction and recycling of lithium resources...*”

Esto resuelve tu pedido de títulos con sentido (7.1b): keywords para clasificar, **frase centroide como subtítulo** para entender.

Dos guardas contra trampas

- **Tope por paper** (`per_paper_cap=2`): sin él, un abstract muy “central” podría monopolizar el resumen entero del cluster.
- El centroide se calcula sobre los vectores de PAPER (los del mapa), no sobre las frases: así el resumen es fiel al mismo espacio que el usuario está viendo.

Ojo / error típico

Las frases top se parecen al centroide... y por eso se parecen ENTRE SÍ: el ranking por centroide tiende a la redundancia (tres frases diciendo lo mismo). Ese es exactamente el problema que ataca MMR en la sub-fase 7.3.

En una frase

Frase centroide = $\cos(\text{frase}, \text{centro del cluster})$, misma matemática que el filtro de relevancia pero para coronar en vez de descartar. En Beauvoir produce definiciones de libro con scores 0.92-0.95. Su debilidad (redundancia) la corrige MMR (7.3).

Apunte 36 — TextRank: PageRank sobre frases (el hub de las frases)

Proyecto Froth · aprender Machine Learning construyendo

La idea: reciclar el algoritmo de Google

PageRank ordenaba la web con una idea circular pero resoluble: *una página importa si páginas importantes la enlazan*. TextRank (Mihalcea & Tarau, 2004) cambia páginas por frases y enlaces por similitud: *una frase importa si frases importantes se le parecen*. Es EXACTAMENTE tu concepto de hub de la Fase 4, aplicado dentro de cada cluster.

Concepto: el grafo de frases y el paseo aleatorio

Nodos = las frases del cluster; peso de la arista = coseno entre sus vectores SPECTER. PageRank se calcula con **iteración de potencias**: imagina un lector saltando de frase en frase, prefiriendo saltos hacia frases similares; con probabilidad $1 - d$ ($d = 0.85$) se teletransporta a cualquiera. Tras iterar

$$r \leftarrow \frac{1 - d}{N} + d W r$$

unas 50 veces, r converge: el tiempo que el lector pasa en cada frase ES su importancia. En `froth/review.py` son 4 líneas de numpy — sin librerías nuevas.

Centroide vs TextRank: no miden lo mismo

- **Centroide** premia lo *típico*: la frase más parecida al promedio.
- **TextRank** premia lo *respaldado*: la frase con más “votos” de consenso, aunque el consenso venga de un subgrupo denso.

En tu corpus (¡corre `python -m froth.review` y júzgalo!):

- Cluster USGS: AMBOS eligen “Minerals Yearbook — These annual publications review...” — cuando dos métricas independientes coinciden, esa frase ES el tema.
- Cluster lepidolita: TextRank prefirió “...the lepidolite flotation process and its challenges are summarized, including poor selectivity...” — frases de REVIEW (resumen los retos del campo) reciben muchos votos porque tocan a todas. Para un literature review, eso es una virtud.
- Cluster geología: TextRank se fue a depósitos exógenos — un subgrupo denso de frases mineralógicas se vota entre sí y captura el ranking. El centroide, anclado a los vectores de los PAPERS, resiste mejor esa trampa.

Ojo / error típico

Los scores no son comparables entre métodos: el coseno vive en 0–1; PageRank reparte una probabilidad total de 1 entre N frases (por eso ves 0.008). Compara RANKINGS, no números. Y ninguno de los dos evita la redundancia — eso es MMR (7.3).

En una frase

TextRank = PageRank sobre el grafo de similitud de frases, resuelto por iteración de potencias. Premia el consenso (bueno para frases-resumen), pero un subgrupo denso puede capturarlo; el centroide premia lo típico. El experto decide cuál lee mejor — o se combinan.

Apunte 37 — MMR: relevante pero distinto (y un objetivo que salió degenerado)

Proyecto Froth · aprender Machine Learning construyendo

El problema que quedaba

El centroide (apunte 35) tiene un defecto estructural: las frases que se parecen al centro se parecen ENTRE SÍ. En tu cluster de geología, las 3 mejores frases tenían similitud mutua de **0.945** — un resumen diciendo lo mismo tres veces.

Concepto: Maximal Marginal Relevance (Carbonell & Goldstein, 1998)

Selección voraz, una frase a la vez. Cada candidata se puntúa como

$$\lambda \cdot \text{relevancia} - (1 - \lambda) \cdot \text{redundancia}$$

donde redundancia = su similitud máxima con lo YA elegido. Con $\lambda = 1$ vuelves al centroide puro; bajando λ compras diversidad pagando relevancia. La primera elegida siempre es la más relevante (no hay nada elegido aún con qué ser redundante).

El tropiezo (que enseña más que el acierto)

Primer intento de elegir λ con datos: “maximizar relevancia – redundancia”. Resultado: recomendó $\lambda = 0.5$... el BORDE de la grilla. Ese objetivo es **degenerado**: bajar λ SIEMPRE reduce la redundancia más rápido de lo que pierde relevancia, así que el “óptimo” se escapa hacia abajo indefinidamente. Un recomendador que siempre elige el extremo no está midiendo nada.

La regla que sí funciona

Mirando tu barrida real (corpus Beauvoir, 12 clusters):

λ	relevancia	redundancia
0.5	0.868	0.758
0.6	0.906	0.829
0.7	0.918	0.860
0.8	0.921	0.873
1.0	0.923	0.884

De 1.0 a 0.7 la relevancia casi no cae (–0.5%) y la diversidad llega gratis; de 0.7 hacia abajo empiezas a pagar de verdad. Regla adoptada (interpretable): **el λ más diverso que conserve $\geq 99\%$ de la relevancia alcanzable**. Tu corpus votó $\lambda = 0.7$ solo — y coincide con el default de la literatura, pero ahora SABEMOS por qué vale para estos datos (regla 9 del proyecto).

Pruébalo tú (teclado en mano)

```
.venv\Scripts\python.exe -m froth.review — al final imprime la barrida y el
antes/después del cluster más redundante: el centroide da 3 frases clónicas de pegmatitas;
MMR mantiene la definición y añade granitos Nb-Ta y el Erzgebirge. Tres ángulos, no un
eco.
```

Ojo / error típico

MMR es voraz: elige lo mejor AHORA sin mirar adelante, porque el óptimo exacto es combinatorio (probar todas las combinaciones de frases). Para resúmenes de 3-5 frases, la aproximación voraz es el estándar y sobra.

En una frase

$MMR = \lambda \cdot \text{relevancia} - (1 - \lambda) \cdot \text{redundancia}$, selección voraz. El primer objetivo para elegir λ salió degenerado (se clavó al borde); la regla final — máxima diversidad conservando el 99% de la relevancia — eligió 0.7 con tus datos. Con esto están las 3 piezas del ranking: típico (centroide), respaldado (TextRank) y diverso (MMR). Sigue ensamblar el draft v2 (7.4).

Apunte 38 — El draft v2: mezclar jueces con Borda y citar por construcción

Proyecto Froth · aprender Machine Learning construyendo

Ensamblar sin inventar

`draft_review_v2()` junta las tres piezas (apuntes 35-37) en un borrador por cluster: una frase ABRE (panorama), 2 frases RESPALDAN (típicas pero diversas), y cada una lleva su cita $[n]$ apuntando al paper del que salió textual. La numeración es global y por orden de aparición; el mismo motor emite el `.bib` para LaTeX/Zotero. Nada se genera: nada puede alucinarse.

El problema del opener y la solución de Borda

El plan era “TextRank abre” (sus frases-panorama son ideales). Pero en geología abrió con la frase de depósitos exógenos — la captura por subgrupo denso del apunte 36. Los scores de centroide (coseno 0-1) y TextRank (probabilidad repartida) NO son comparables, así que no se pueden promediar... pero los *rankings* sí.

Concepto: conteo de Borda (1770, elecciones; hoy, rank aggregation)

Cada método ordena las frases y la posición es el “voto”: $1^{\circ} = 0$ puntos, $2^{\circ} = 1$, etc. Gana la frase con MENOR suma de posiciones. Para secuestrar el opener habría que engañar a los DOS jueces a la vez: el subgrupo denso pierde porque el centroide rankea bajas sus frases. Con tus datos: geología pasó de exógenos a composición química de pegmatitas; lepidolita conservó su apertura panorámica.

Fidelidad: rechazar, no recortar

El primer draft abrió lepidolita con “*In addition, ...*” — un conector que apunta a texto inexistente. Dos arreglos posibles: RECORTAR el conector (modifica la cita textual: prohibido por la regla de fidelidad del proyecto) o RECHAZAR la frase y dejar que Borda promueva la siguiente. Se rechaza: el filtro `_self_contained()` ahora también veta conectores de discurso de dos palabras (“In addition”, “On the other hand”, “In contrast”...). La siguiente elegida fue “*Efficient flotation of lepidolite is crucial...*” — mejor apertura, cero manipulación del original.

Ojo / error típico

El $\dagger$ del draft marca frases donde centroide y TextRank coinciden en el top-3 (consenso = sello de confianza). Si una sección no tiene $\dagger$, no está mal — solo significa que los dos jueces vieron virtudes distintas.

Pruébalo tú (teclado en mano)

```
.venv\Scripts\python.exe -m froth.review genera 3_Resultados/review_draft_v2.md
```

(42 refs trazables en Beauvoir) y su `.bib`. Abre el `.md` y verifica una cita: busca el $[n]$, ve a References, y comprueba que la frase existe en el abstract de ese paper. Trazabilidad auditable.

En una frase

Draft v2 = Borda elige el opener (engañar a dos jueces es más difícil que a uno), MMR pone el respaldo diverso, cada frase va textual con cita [n] + BibTeX. Frases con conectores colgantes se RECHAZAN (nunca se recortan): la fidelidad manda.

Apunte 39 — El LLM como pulidor verificado (nunca como autor)

Proyecto Froth · aprender Machine Learning construyendo

La jerarquía de confianza

El draft v2 es extractivo: verdad garantizada, prosa regular. Un LLM escribe prosa excelente... sin garantía de verdad. La arquitectura de la fase 7.6 saca lo mejor de ambos SIN heredar el riesgo:

Concepto: pulir $\neq$ generar

El LLM local (Ollama) recibe UNA instrucción estrecha: reescribir el párrafo de una sección para que fluya, sin añadir ni quitar afirmaciones, manteniendo cada cita $[n]$ exactamente una vez. No investiga, no opina, no cita: PULE frases que ya existen. El conocimiento viene del corpus; el LLM solo aporta gramática.

La puerta de verificación (el corazón del diseño)

Confiar no basta: cada sección reescrita pasa por `_verified()` en `froth/polish.py`:

- **Censo de citas**: las citas $[n]$ del texto pulido deben ser EXACTAMENTE las del original (ni una menos = borró evidencia; ni una más = inventó).
- **Banda de longitud** ($0.5\text{--}1.7\times$): fuera de ahí, o resumió de más (perdió afirmaciones) o divagó (probablemente añadió).
- Si CUALQUIER chequeo falla $\rightarrow$ la sección conserva su original extractivo, en silencio. El draft solo puede mejorar, jamás mentir.

Probado sin servidor: la puerta rechazó cita borrada, cita inventada y texto encogido; y sin Ollama corriendo el draft queda intacto (fallo suave, como los conectores de fuentes).

Por qué LOCAL (Ollama) y no una API

Decisión tuya de la Fase 6 que aquí paga: gratis (sin cuotas), privado (tu review no sale de tu PC), offline, y diferenciador — Elicit/SciSpace/Consensus dependen de OpenAI. Tu RTX 5060 corre un modelo 8B cómodamente. Setup en `docs/OLLAMA_SETUP.md` (un instalador + `ollama pull llama3.1:8b`, ~ 5 GB, una sola vez).

Ojo / error típico

Temperatura 0.2 (casi determinista): para pulir no quieres creatividad, quieres obediencia. Y el pulido es una DESCARGA SEPARADA — el extractivo sigue siendo el default visible. La cadena de confianza: mapa medido $\rightarrow$ frases reales $\rightarrow$ prosa pulida verificada. Cada eslabón auditable.

En una frase

El LLM local reescribe párrafos ya contruidos y una puerta automática (censo de citas + banda de longitud) descarta cualquier reescritura sospechosa, sección por sección. Prosa de LLM con la trazabilidad del extractivo — o nada.

Apunte 40 — Títulos de cluster con sentido: KeyBERT (no oraciones recortadas)

Proyecto Froth · aprender Machine Learning construyendo

El problema (lo viste tú)

La v1 de los subtítulos tomaba la *oración* más central del cluster y la cortaba a 12 palabras. Resultado: encabezados como “Considering both...”, “In the present st...” — fragmentos que insinúan el tema pero se sienten recortados, como una biblia a medio leer. Y las keywords viejas (c-TF-IDF: “dpa, dda, lepidolite”) tampoco: clasifican bien pero *no comunican*.

Cómo lo resuelve el campo: KeyBERT

El estándar (KeyBERT, y la capa de representación de BERTopic) no recorta oraciones: extrae **keyphrases** cortas y las rankea por SIGNIFICADO.

Concepto: KeyBERTInspired en 3 pasos

1. **Candidatos:** n-gramas de 2–3 palabras del texto del cluster (con el mismo vectorizador endurecido de `cluster.py`: sin dígitos, elementos químicos salvados).
2. **Ranear:** embeber cada candidato con SPECTER y medir su coseno al *centroide* del cluster.
3. **Diversificar:** MMR para que las frases elegidas no sean casi iguales.

Lo elegante: son las MISMAS piezas que Froth ya tenía (SPECTER + coseno + centroide + MMR de la Fase 7). Por eso se hizo a mano, sin dependencia nueva — el patrón del proyecto: construir cada pieza para entenderla.

Dos tropiezos que enseñan (tus datos reales)

El primer intento sacó títulos con ruido de OCR (“limu”, “nmic”, “cgfs”) y genéricos (“ore flotation” donde debía decir “lepidolite flotation”). Dos arreglos:

- **min_df $\geq$ 2:** exigir que la frase aparezca en VARIOS papers del cluster. La basura de OCR vive en un solo abstract $\rightarrow$ fuera.
- **Score contrastivo** (la idea de c-TF-IDF, pero en el espacio de embeddings):

$$\text{score} = \cos(\text{frase}, \text{centroide}_c) - 0.4 \cos(\text{frase}, \text{centroide}_{\text{global}})$$

Premia lo que está cerca de ESTE cluster y lejos del corpus entero. Así “lepidolite flotation” (específico) le gana a “ore flotation” (genérico, cerca del centro de todo).

Ojo / error típico

El keyphrase más cercano al centroide NO es el mejor título: suele ser el más genérico. La especificidad (lejos del promedio global) es lo que hace que un título *diga* algo. Misma lección que el h-index y el filtro de relevancia: la señal está en el contraste, no en el valor absoluto.

Pruébalo tú (teclado en mano)

El título corto va al encabezado; la oración centroe completa se guarda en la columna **summary** y aparece en el **hover** — corto arriba, detalle al pasar el mouse, justo como lo pediste.

En una frase

Títulos de cluster = keyphrases de 2–3 palabras extraídas por KeyBERT (candidatos + SPECTER + coseno al centroe + MMR), con `min_df` contra el ruido y un score contrastivo (cluster – global) para preferir lo específico. Cortos, con sentido, sin recortar oraciones; la frase completa vive en el hover.

Apunte 41 — El veredicto del re-fine-tune: más datos NO mejoró

Proyecto Froth · aprender Machine Learning construyendo

La hipótesis (tu duda del learning log)

El fine-tune v1 (specter-mineral, 1.290 papers) ganaba 14/22 en DBCV pero castigaba la periferia (hasta -0.43). Hipótesis natural: con **3× más datos** (re-cosecha con keys $\rightarrow$ 4.045 papers) el modelo v2 debería generalizar mejor y dejar de penalizar los topics lejanos. Lo pusimos a prueba, medido.

El experimento limpio

Entrenamos v2 (4.045 pares título $\leftrightarrow$ abstract, MNRL, 249s en tu RTX 5060) y comparamos **los tres modelos sobre los MISMOS 22 corpus nuevos** por DBCV: base (SPECTER genérico), v1 (fine-tune viejo) y v2 (fine-tune grande).

	gana a base	delta medio
v1 (1.290 papers)	16/22	+0.093
v2 (4.045 papers)	14/22	+0.069
v2 vs v1 directo	10/22	-0.024

Concepto: el resultado: escalar datos NO ayudó (y hasta empeoró un poco)

v2 gana a base MENOS veces que v1 (14 vs 16) y con menor margen. Cabeza a cabeza, v2 le gana a v1 en solo 10/22 con delta medio negativo. **El fine-tune pequeño y curado fue igual o mejor que el grande.**

Por qué (la lección de verdad)

La re-cosecha 3× trajo más papers, pero también más DIVERSOS y RUIDOSOS (colisiones de términos multi-dominio como “attrition”/“autogenous”, más subdominios). El fine-tuning auto-supervisado arrastra el modelo hacia el *centro de masa* del corpus de entrenamiento (apunte 31); al agrandar el corpus con ruido, ese centro se mueve hacia lo genérico y la especialización se DILUYE.

Ojo / error típico

En fine-tuning auto-supervisado, la CALIDAD y coherencia del corpus importa más que la cantidad. “Más datos” es un reflejo, no una ley. La varianza sigue altísima: v2 va de $+0.47$ (escorias XCT) a -0.51 (finos de silicato de hierro) — el fine-tune es una apuesta por-topic, no una mejora uniforme.

Qué queda en producción

Nada cambia: producción sigue con el SPECTER **base** (el más estable across topics). Los fine-tunes son el RESULTADO DE INVESTIGACIÓN. v1 se conserva (models/specter-mineral-v1); v2 queda como referencia. Evidencia completa en 2_Datos/model_comparison_3way.csv.

En una frase

Re-fine-tune con $3\times$ datos: v2 gana a base 14/22 (v1 ganaba 16/22), y pierde contra v1 cabeza a cabeza (delta medio -0.024). Más datos NO mejoró — el corpus grande y ruidoso diluyó la especialización. Lección de tesis honesta: en fine-tuning auto-supervisado, la curación del corpus $>$ el tamaño. “No mejora” también es resultado.

Apunte 42 — Cuando el programa no se cae sino se CUELGA: watchdogs y subprocessos

Proyecto Froth · aprender Machine Learning construyendo

El accidente real (madrugada del 14 de julio)

Lanzamos la re-cosecha sin techo de los 22 topics. El batch iba perfecto: 18 corpus gigantes (4.000–16.681 papers cada uno) en unas 3 horas... y de repente, silencio. A la mañana siguiente: **9 horas congelado** en el topic #20, sin error, sin mensaje, sin avanzar. El proceso seguía “vivo” (77 MB de RAM, cero CPU), esperando una respuesta de red que jamás llegó.

La lección central: un cuelgue NO es un crash

Nuestro `run_topic` ya era “a prueba de fallos”: envolvía todo el pipeline en `try/except`, y cada fallo se convertía en una fila **FAILED** del reporte para seguir con el siguiente topic. Sonaba blindado. No lo estaba.

Concepto: crash vs. cuelgue

Un **crash** lanza una excepción: `try/except` la atrapa y la vida sigue. Un **cuelgue** (hang) no lanza NADA: el programa se queda esperando para siempre (un socket muerto, un cómputo desbocado, un driver bloqueado). Ningún `try/except` del mundo interrumpe una espera infinita — para eso hace falta alguien MIRANDO DESDE AFUERA con un reloj: un **watchdog**.

Y como los topics corrían en serie, un solo cuelgue congeló el batch entero. La probabilidad no era despreciable: 22 topics $\times$ 4 APIs $\times$ miles de requests.

El arreglo: proceso hijo + reloj de pared

Cada topic corre ahora en un **subproceso** propio (`python -m froth.batch --one "<título>"`) vigilado por el padre con `subprocess.run(..., timeout=TOPIC_TIMEOUT_S)` (2.400 s, generoso: el topic legítimo más lento tardó 38 min). Si el hijo se pasa del reloj, Windows lo MATA, el padre escribe **FAILED: timed out** y sigue con el siguiente. El cuelgue pasó de “catástrofe silenciosa de 9 horas” a “una fila fea en el CSV”.

De paso acotamos los dos cómputos que podían desbocarse de verdad: el barrido DBCV para la granularidad automática es $\mathcal{O}(n^2)$ y se repite 18 veces — sobre 16.000 puntos son horas; ahora, pasado un tope (`DBCV_SUBSAMPLE_CAP = 5.000`) se barre sobre una submuestra aleatoria (la ELECCIÓN de granularidad es estable; el costo no), y el clustering final sí usa todos los puntos. Y el rellenado de abstracts, el único bucle de red sin presupuesto, recibió su límite de tiempo.

Bonus 1: el plan B de Beauvoir

El topic “Recovery of by-products of lithium from the a rare-metal granite” cosechó **0 papers**: sus términos derivados giraban sobre “a rare-metal granite”, demasiado nicho para las APIs. Arreglo: si la cosecha vuelve VACÍA, se reintenta UNA vez con una query amplia de alto recall (las primeras palabras de contenido del título: `recovery by-products lithium`). Resultado real: de 178 papers viejos a **4.937**.

Bonus 2: el enemigo externo (WDAC en olas)

La política corporativa de control de aplicaciones (WDAC) bloquea a ratos las DLL nativas de `scipy/hdbscan/torch` (“Una directiva de Control de aplicaciones bloqueó este archivo”). Medido en vivo: dejó trabajar 19 minutos a un hijo y 5 minutos después bloqueó a los otros dos en el import. Contra una política corporativa no se pelea: se ESPERA con paciencia automatizada — un sondeo barato (`python -c "import hdbscan, umap, torch"`) cada 5 minutos, y cuando la ola pasa, se lanza el trabajo pesado. El orquestador padre, además, solo importa `config` + `pandas`: sin DLLs científicas, WDAC no puede tumbarlo.

Ojo / error típico

Tres capas distintas de defensa, porque son tres fallos distintos: `try/except` atrapa *crashes*, el watchdog con timeout mata *cuelgues*, y el sondeo de ventanas espera *bloqueos del entorno*. Confundirlas es creer que un chaleco salvavidas te protege de un rayo.

En una frase

El batch se congeló 9 horas porque un cuelgue no lanza excepción y nada lo vigilaba. Arreglo en tres capas: subproceso por topic con timeout de pared (watchdog), cómputo acotado (DBCV por submuestra), y sondeo de ventanas WDAC. Resultado: 22/22 topics con estado claro y ~169.000 papers cosechados; Beauvoir pasó de 178 a 4.937 con el plan B de términos, a synthetic-ore topic de 237 a 9.723. Un sistema robusto no es el que nunca falla: es el que convierte cualquier fallo en una fila del reporte y sigue.

Apunte 43 — La puerta de relevancia: separar el grano de la paja en 190.000 papers

Proyecto Froth · aprender Machine Learning construyendo

Glosario en palabras simples (léelo primero)

- **Vector:** la “huella digital” numérica de un paper — una lista de 768 números que captura su significado. Papers que hablan de lo mismo tienen huellas parecidas (apunte 05).
- **Coseno:** el medidor de parecido entre dos huellas. Va de -1 a 1 : cerca de 1 = hablan de lo mismo; cerca de 0 = nada que ver.
- **Centroide:** el “promedio” de un grupo de vectores — su centro de gravedad, como la ley media de un lote de mineral.
- **Umbral:** el valor de corte. Encima: el paper entra al mapa. Debajo: fuera.
- **Bimodal:** distribución con DOS jorobas. Si los buenos hacen una y los malos otra, el valle entre ambas es el corte natural.
- **Hub / cluster:** grupo de papers muy parecidos entre sí — un subtema.

El problema que TÚ encontraste

Con la cosecha sin techo (190.000 papers), abriste el mapa de tailings y viste “*cosas hasta de recursos humanos, en colores*”. Tu hipótesis fue exacta: ninguna palabra falló al vectorizarse — falló **la puerta** que decide qué entra al mapa.

La puerta vieja: relevancia = $\cos(\text{paper}, \text{TÍTULO del topic})$, con umbral fijo 0.55. Medido en tailings: aceptaba el **93%** de 16.681 papers, incluyendo clamidiosis en koalas (por “iron” biológico), “event management” (por “management”) y “Circular Economy in SMEs” (por “circular economy”). Una frase corta con imanes genéricos no puede distinguir facetas. Además, el techo viejo de 25 resultados por término era un filtro de precisión ACCIDENTAL: al quitarlo (decisión correcta: cobertura es la misión), entró la cola profunda de cada término.

Mini-ejemplo de juguete (haz las cuentas conmigo)

Huellas de solo 2 números (en vez de 768). El título apunta a $T = (1, 0)$. Tres papers:

	vector	cos con el título
A: flotación de relaves	(0.98, 0.20)	0.98
B: gestión de personal	(0.60, 0.80)	0.60
C: koalas	(0.50, 0.87)	0.50

Con umbral 0.55, ¡B entra al mapa! ($0.60 > 0.55$). Ese es tu hub de recursos humanos.

La **puerta contrastiva** pregunta algo mejor: ¿te pareces a mi tema MÁS de lo que te pareces a lo genérico?

$$v_2 = \cos(\text{paper}, \text{núcleo}) - 0.4 \cdot \cos(\text{paper}, \text{global})$$

donde núcleo $\approx A$ (los papers más pegados al título) y global = promedio de todos $\approx (0.74, 0.67)$ normalizado. Cuentas:

$$v_2(A) = 1.00 - 0.4 \cdot 0.86 = \mathbf{0.66} \quad v_2(B) = 0.75 - 0.4 \cdot 0.98 = \mathbf{0.36}$$

B queda lejos de cualquier corte razonable. Con 768 números y 16.681 papers, esto mismo produjo una distribución **bimodal** con valle en $+0.399$ — y el umbral se elige en ESE valle, por corpus (perilla calculada, no constante fósil).

Concepto: la idea contrastiva

Es el mismo truco que arregló los títulos KeyBERT (apunte 40): premiar cercanía al núcleo Y castigar cercanía a la masa genérica. Lo genérico compartido deja de contar como mérito.

Tropiezo 1: la puerta por-paper aplanó los mapas

La primera versión juzgaba paper por paper... y tailings pasó de 115 subtemas a **2** (a CUALQUIER granularidad — lo confirmó el barrido DBCV). ¿Por qué? Al quedarte solo con “los parecidos al núcleo”, la población superviviente es una banda homogénea: una bola sin valles internos que HDBSCAN no puede subdividir. Ganamos precisión, destruimos la ESTRUCTURA — que es la mitad del producto.

El arreglo salió de tu propia frase (“un hub de RRHH, *en colores*”): la basura llega en **hubs coherentes**, así que la puerta debe juzgar HUBS: (1) mapear TODO, (2) puntuar cada cluster por el v_2 promedio de sus miembros, (3) botar clusters enteros bajo el valle de esas medias, (4) el ruido suelto se juzga individual. Medido: los hubs de física de partículas, astronomía, RRHH y koalas murieron completos; el de morteros con relaves no perdió ni un paper; 325 subtemas del dominio intactos.

Tropiezo 2: tu doble ciego destapó un sesgo

Juzgaste 40 casos de frontera a ciegas: 62% de acuerdo, pero con dirección — rescataste 11 papers que la máquina botaba, casi todos de **economía circular**... que está EN el título de tu tesis. Causa: el centro global apunta en parte hacia el propio tema (el corpus está lleno de él), así que un paper alineado con el título recibía premio del núcleo Y castigo del global: doble penalización por su propia identidad. Arreglo: **proyectar fuera** del global la dirección del núcleo antes de castigar. Verificado: “Towards Circular Economy Metrics” volvió; koalas y el paper del queso (“Iron Ages”... de la Edad del Hierro, no del mineral) siguen fuera.

Honestidad de cierre: re-medido tu acuerdo contra la puerta final, quedó en 60% — prácticamente igual. El fix ganó los casos que te importaban (los de economía circular), pero la frontera de ± 0.04 del valle es ambigua HASTA para el experto (tú dejaste pasar el queso por el “Iron”). No se afina más: sería sobreajustar a 40 casos.

Ojo / error típico

El valor del gate está en el nivel de HUBS (basura coherente muere entera, estructura del dominio intacta), no en decorar el borde. Ninguna frontera unidimensional es perfecta; el flujo correcto es máquina propone → experto valida → la perilla se recalibra donde el experto tiene razón sistemática (economía circular sí, queso no).

En una frase

Puerta vieja: cos con el título + umbral fijo = 93% aceptado, koalas en colores. Puerta nueva: score contrastivo (núcleo vs. global SIN la dirección del tema) → bimodal → valle POR corpus → juicio por HUBS para preservar estructura. Resultado: 22/22 topics limpios con su propio umbral, master de 77.636 papers únicos relevantes. Tres lecciones: (1) contrastar contra lo genérico separa lo que el parecido a secas no puede; (2) filtrar individuos puede destruir la estructura colectiva — juzga al grupo cuando la señal es grupal; (3) el juez humano encuentra los sesgos que la métrica no ve — y también tiene los suyos.

Apunte 44 — Tripletas de citación: entrenar como SPECTER — y el veredicto invertido

Proyecto Froth · aprender Machine Learning construyendo

Glosario en palabras simples

- **Fine-tuning:** tomar un modelo ya entrenado (SPECTER) y darle un “curso de especialización” con datos de TU dominio.
- **Tripleta:** un ejemplo de entrenamiento con 3 piezas: *ancla* (un paper), *positivo* (uno que debe quedar CERCA) y *negativo* (uno que debe quedar LEJOS).
- **Hard negative:** un negativo DIFÍCIL — se parece al ancla pero no es lo que buscas. Enseña mucho más que un negativo obvio.
- **Época:** una pasada completa por todos los ejemplos de entrenamiento.
- **DBCV:** nota de calidad de un clustering por densidad (−1 a 1): premia grupos densos por dentro y bien separados entre sí.
- **Doble ciego:** evaluar sin saber qué opción es cuál, para que el juicio no se contamine.

La pregunta que hiciste

“¿Deberíamos poder mejorar el fine-tune, o SPECTER es perfecto?” La respuesta corta: SPECTER no es perfecto, pero nuestros fine-tunes anteriores usaban un método débil. v1 y v2 entrenaban con pares (título ↔ abstract): eso solo enseña “los títulos van con sus abstracts”, y arrastra todo al centro de masa del corpus — por eso v2 (3× datos) NO superó a v1 (apunte 41). El SPECTER original se entrena distinto: con **tripletas de citación** — si el paper A cita al B, B debe quedar cerca de A; un paper que A no cita debe quedar lejos.

Mini-ejemplo de juguete

Ancla: “Flotación de micas con colectores catiónicos”. Positivo: el paper de colectores que ELLA CITA. Hard negative: un paper citado por el positivo pero NO por el ancla (a dos saltos: pariente del pariente, pero no tuyo). En cada paso, el entrenamiento MIDE las distancias y empuja: positivo un poco más cerca, negativo un poco más lejos. Repite 100.000 veces y el espacio entero se reorganiza según quién-cita-a-quién.

La cosecha sin techo hizo posible esto a escala: de 833 tripletas (corpus viejo) a **150.885** (100.190 con hard negative) — 181× más señal, porque con 190.000 papers muchas referencias caen DENTRO del corpus.

El entrenamiento (y sus perillas medidas)

Primer intento: 2 épocas × secuencias de 512 tokens = 17 horas estimadas en tu RTX 5060 (3,2 s/paso, medido). Decisión tuya: 1 época + 256 tokens (los abstracts ~200 tokens caben casi enteros) + checkpoints cada 1.000 pasos (lección del apunte 42: una interrupción cuesta minutos, no todo). Resultado: 3,4 pasos/segundo — terminó en ~45 minutos. El modelo: `specter-mineral-v3`.

El veredicto en dos rondas

Ronda 1 — la métrica (DBCV, 22 corpus limpios, muestra fija de 1.500):

	gana a base	delta medio
v1 (pares, chico)	14/22	+0.013
v2 (pares, grande)	9/22	−0.022
v3 (citación)	12/22	+ 0.023
v3 vs v2 directo	13/22	+0.046

Tu pregunta quedó respondida: el MÉTODO correcto gana donde la CANTIDAD falló (v2 pierde contra base; v3 es el único con mejora media clara). Y un hallazgo extra: en los corpus sucios viejos v1 ganaba 16/22 con +0.093; al limpiar, TODAS las ventajas se encogieron — parte de la “mejora” anterior era saber agrupar la basura.

Ronda 2 — el experto (tú, doble ciego, 4 mapas anónimos): rankeaste B>D>C>A... que al revelar la llave resultó ser **v1 > v2 > base > v3**. ¡El ranking casi ESPEJO de la métrica! El favorito del DBCV (v3, 0.377) fue tu último lugar.

Concepto: geometría ≠ narrativa

El DBCV mide si los grupos son densos y separados (geometría). Tú mides si el mapa CUENTA la historia del campo (narrativa). v3 aprendió de citas, así que agrupa *comunidades de citación* — quién cita a quién: escuelas, métodos, revistas — que forman blobs preciosos para la métrica pero no siempre coinciden con TEMAS legibles. v1, con su ajuste suave, conserva la organización semántica que un experto lee.

Las decisiones (tuyas, como juez)

- **Producción** → **v1**: lo elegiste a ciegas DOS veces (2026-07-12 y 2026-07-16) y gana a base 14/22. Tu instalación usa v1 local; la versión pública cae automáticamente a SPECTER de fábrica (sin fricción, sin descargas manuales).
- **v3 no se bota** — **se re-propósito**: su espacio “citacional” es justo la señal que necesita el futuro modo semilla (“papers similares a ESTE”, estilo ResearchRabbit): ahí las comunidades de citación SON lo que buscas.

Ojo / error típico

Si solo hubiéramos mirado la métrica, habríamos puesto en producción el modelo que el experto lee PEOR. Si solo hubiéramos mirado al experto, no sabríamos POR QUÉ v3 agrupa raro (ni que es oro para otra tarea). Máquina propone, experto valida, y las dos miradas juntas valen más que cualquiera sola.

En una frase

La receta real de SPECTER (tripletes de citación + hard negatives) aplicada a TUS 190.000 papers: 150.885 tripletes, 45 min de GPU, v3. La métrica lo corona (+0.023 sobre base, único positivo claro; método > cantidad confirmado); tu doble ciego lo manda de último y corona a v1 — ranking espejo. Lección doble: (1) el método de entrenamiento importa más que el volumen de datos; (2) la calidad geométrica no es calidad narrativa: elige el modelo según la TAREA (v1 para mapas legibles, v3 como candidato para similitud por citación). “No mejora para esto” también es resultado — y encontrarle su tarea correcta, también.

Apunte 45 — Zoom semántico: por qué 10.921 papers caben en pantalla y 1.772 no

Proyecto Froth · aprender Machine Learning construyendo

Glosario en palabras simples

- **Renderizar**: dibujar en la pantalla. Cada punto, cada línea, cada letra que ves alguien tuvo que pintarla.
- **DOM**: la lista de objetos que el navegador guarda para cada cosa dibujada en una página. Si dibujas 10.000 círculos como objetos, el navegador guarda 10.000 fichas y las revisa continuamente.
- **SVG**: forma de dibujar donde CADA figura es un objeto del DOM. Cómoda (puedes darle clic a cada una) pero cara: miles de objetos ahogan al navegador.
- **WebGL**: forma de dibujar que le pasa los puntos a la **tarjeta gráfica** (la GPU). La GPU pinta millones de puntos a la vez porque para eso está hecha. No guarda una ficha por punto.
- **GPU**: el chip que dibuja. Tu portátil tiene una RTX 5060. Hace una cosa muy bien: la misma operación sobre miles de datos *en paralelo*.
- **Hilo principal** (*main thread*): el único carril por donde el navegador atiende TODO: tus clics, la rueda del ratón, los cálculos, el dibujo. Si algo lo ocupa, la página deja de responder aunque no esté “rota”.
- **Fotograma** (*frame*): una imagen de la pantalla. Para que algo se sienta fluido hacen falta unos 60 por segundo, o sea ≈ 16 ms para cada uno.
- **Física de grafo**: simulación donde los nodos se repelen y las aristas tiran como muelles, hasta que el dibujo se “acomoda” solo.
- **Percentil**: si ordenas 201 números de menor a mayor, el percentil 90 es el valor que deja por debajo al 90% de ellos. Sirve para decir “los grandes” sin inventarse un número.
- **Centroide**: el punto medio de un grupo. Donde colocamos su nombre.

El problema, tal como apareció

Subimos el tope de papers de la vista *Network* de 400 a 2.500 y quedaron 1.772 nodos en pantalla. Yo escribí “no se rompe nada”. Tu pantallazo dijo otra cosa: una bola de pelo congelada, sin zoom ni desplazamiento.

Y sin embargo, en la misma sesión, la vista *Atlas* dibujó **10.921 papers** y respondía. Casi seis veces más puntos, y fluida. Ese contraste es el apunte entero.

Concepto: Dibujar no es simular

La Network usa **física**: en cada fotograma recalcula las fuerzas entre nodos y, cuando mueves el ratón, comprueba nodo por nodo cuál está debajo del cursor. Con 1.772 nodos eso llena los 16 ms del fotograma y no queda tiempo para atender la rueda. El zoom no estaba roto: **el carril estaba ocupado**.

El Atlas no simula nada. Las posiciones ya están calculadas de antes (UMAP, apunte 08) y guardadas. Solo las manda a la GPU y esta las pinta. Por eso 10.921 le cuestan menos que 1.772 con física.

Ojo / error típico

Mi error, con todas las letras. Cuando dije “no se rompe nada” yo había medido dos cosas: cuánto tarda Python en *generar* el HTML, y cuánto tarda vis.js en *asentar* el dibujo. Las dos baratas. **No medí la interacción**, que es justo lo que se rompió.

La lección, que vale para todo el proyecto: **que algo sea rápido de PRODUCIR no dice nada sobre si es fluido de USAR**. Son dos medidas distintas y hay que tomar las dos.

La otra mitad: 201 nombres no caben

Arreglado el motor de dibujo, aparece el problema humano. El Atlas pintaba los puntos con un color por subtema y nada más: bonito, y mudo. Para saber qué era cada mancha había que irse a la leyenda lateral y cruzar colores a ojo.

La solución obvia (poner el nombre de cada subtema sobre su centroide) no funciona: en el corpus de LIBS/FTIR hay **201 subtemas**. Doscientos un rótulos a la vez son una pared de texto, no un mapa.

Mini-ejemplo de juguete (antes de la teoría)

Imagina un mapa de solo 5 territorios, con este tamaño en papers:

Territorio	A	B	C	D	E
Papers	100	60	30	12	8

Ahora piensa en Google Maps. Muy lejos ves “España”. Te acercas y aparece “Madrid”. Más cerca, “Chamberí”. Más cerca todavía, el nombre de la calle. **Lo que se muestra cambia con la distancia**, no solo el tamaño de las letras. Eso es zoom semántico.

Aplicado a nuestros 5 territorios: desde lejos solo A; te acercas y aparecen A y B; más cerca A, B, C; encima del todo, los cinco. La pregunta interesante es **dónde pones los cortes**, y ahí es donde no vale inventarse un número.

Concepto: Zoom semántico (Shneiderman, 1996)

La regla clásica de visualización dice: *“vista general primero, luego zoom y filtro, y el detalle solo cuando se pide”*. Traducido a Froth: primero unos pocos territorios grandes con nombre, y los subtemas pequeños solo cuando te acercas a su zona. Nunca los 201 de golpe.

Los cortes salen de TUS datos (la regla de siempre)

Un umbral inventado (“muestra los que tengan más de 50 papers”) funcionaría en un corpus y sería absurdo en otro. Así que los cortes se calculan con los **percentiles del tamaño de los subtemas de ese corpus concreto**: percentil 90 (los grandes de verdad), 75, 50, y 0 (todos).

Dos corpus reales tuyos, con los números que salieron:

Corpus	Papers	Subtemas	Cortes calculados (p90/p75/p50/p0)
Beauvoir (tu tesis)	3.867	96	54 / 35 / 19 / 7
LIBS + FTIR	10.921	201	76 / 37 / 20 / 7
Economía circular	4.946	69	142 / 69 / 33 / 7

Fíjate en la tercera fila: economía circular tiene MENOS papers que LIBS pero su corte de “satélite” es casi el doble (142 frente a 76), porque sus subtemas son más gordos y menos numerosos. Un número fijo habría dejado ese mapa vacío o saturado. El corpus decide.

Lo que ves al acercarte, medido en el navegador sobre los 10.921 papers:

Nivel	Metáfora	Corte	Nombres en pantalla
1	Satélite	≥ 76	21
2	Regional	≥ 37	52
3	Local	≥ 20	106
4	Calle	≥ 7	201

Y todavía hay una segunda red de seguridad: aunque en el nivel 4 se manden los 201 rótulos, deck.gl aplica un **filtro de colisión** que descarta los que se pisarían, dejando ganar al territorio más grande. Los 201 solo pueden aparecer juntos si de verdad caben.

Pruébalo tú (teclado en mano)

Abre el Atlas de un corpus grande y haz esto en orden:

1. Sin tocar nada, cuenta los nombres que ves. Deberían ser pocos y todos de territorios gordos.
2. Acerca la rueda dos o tres pasos sobre una zona. Verás aparecer nombres nuevos, y solo en la zona donde estás mirando.
3. Pulsa un subtema en la leyenda lateral: la cámara vuela hasta él.
4. Pulsa **“Whole map”** para volver a la vista general.

Si en el paso 1 ves una pared de texto, el cálculo de percentiles no se está aplicando: avísame y lo miramos.

El tropiezo que enseña más que el éxito

El plan original para “clic en un subtema” era distinto y más ambicioso: recalculan un UMAP **solo con los papers de ese subtema**, para revelar su estructura interna, que el UMAP global aplana. El plan decía “unos 2 segundos, y se puede cachear”.

Antes de construirlo, lo medí: **29,2 segundos** para 500 papers, con la función real (`cluster.reduce_to_2d`). No 2 segundos. Casi treinta.

Con 96 subtemas en tu corpus de tesis, explorar el mapa habría sido esperar medio minuto por cada clic. Inaceptable. Así que el clic hace algo mucho más humilde: **vuela la cámara a las coordenadas que ya existen**. Cero cálculo, cero espera.

Ojo / error típico

La cifra del plan era mía, y era falsa. Ese “ $\sim 2s$ ” no venía de ninguna medición: lo escribí yo por intuición y quedó en el plan como si fuera un dato. Si no lo llego a medir antes de programar, habríamos construido una función inservible y lo habrías descubierto tú, usándola.

Regla: **una cifra en un plan no es un dato hasta que alguien la mide.** Y las mediciones se hacen ANTES de construir, no después.

Se pierde algo real, y conviene decirlo: la sub-estructura interna de cada subtema sigue sin verse. Queda anotado como posible mejora futura (calcularla una sola vez, en el momento de procesar el corpus, no al hacer clic).

La división del trabajo que salió de aquí

En vez de intentar que una vista lo hiciera todo, cada una hace lo que sabe hacer:

	Network	Atlas
Motor	vis.js con física	deck.gl / WebGL (GPU)
Tope de papers	400	el corpus entero
Qué muestra	quién se parece a quién	el territorio completo
Para qué sirve	leer el núcleo más citado	orientarte y elegir dónde mirar

Bajar la Network de 2.500 a 400 **no es una regresión**: es reconocer que la física sirve para ver relaciones entre pocos nodos, y estorba para ver un continente. Si 400 te sabe a poco, el número está en la configuración y se sube sin tocar código; pero la fluidez solo la puede juzgar tu máquina, no la mía.

En una frase

Dibujar en la GPU (WebGL) escala; simular física por fotograma, no. Y cuando hay más nombres que sitio, se muestran por niveles de zoom, con los cortes calculados a partir de los percentiles de TU corpus, no de un número inventado.

Para saber más

- Shneiderman, B. (1996), *The Eyes Have It*: el origen del “overview first, zoom and filter, then details-on-demand”.
- nightingaledvs.com/how-to-visualize-a-graph-with-a-million-nodes
- cambridge-intelligence.com/visualizing-graphs-webgl
- El código: `froth/atlas.py` (funciones `minCountForZoom`, `makeLabels` y `flyTo`).

Apunte 46 — Por qué un programa tarda en abrir: importar tarde y no calcular dos veces

Proyecto Froth · aprender Machine Learning construyendo

Glosario en palabras simples

- **Importar:** cuando tu programa dice `import umap`, Python va al disco, busca esa librería, la lee entera y la deja lista en memoria. Eso tarda. Y ocurre *aunque nunca llegues a usarla*.
- **Librería:** código que escribió otra gente y tú reutilizas. UMAP, HDBSCAN y PyTorch son librerías.
- **Nivel de módulo:** las líneas que están pegadas al margen izquierdo de un archivo `.py`. Se ejecutan en cuanto alguien importa ese archivo, sin que nadie llame a nada.
- **Import perezoso** (*lazy import*): mover el `import` DENTRO de la función que lo necesita. Así el coste se paga la primera vez que usas esa función, no al abrir el programa.
- **Caché:** guardar un resultado ya calculado para no repetirlo. La palabra se pronuncia “cash”.
- **Caché en memoria RAM:** se guarda mientras el programa está vivo. Si cierras el programa, se pierde.
- **Caché en disco:** se guarda en un archivo. Sobrevive a cerrar el programa, a apagar el ordenador y a volver mañana.
- **Clave de caché:** la “etiqueta” con la que guardas el resultado. Si la etiqueta cambia, el resultado guardado ya no vale y hay que recalcular.

El problema, tal como apareció

Batman escribió: “*el tiempo de espera es mucho*”. Frase corta, diagnóstico cero. La regla de esta casa es no adivinar: se mide.

Se cronometró cada pieza del arranque de Froth por separado. Esto salió:

Pieza del arranque	Segundos
importar <code>umap</code> + <code>hdbscan</code>	24,8
importar <code>torch</code> + <code>sentence_transformers</code>	19,7
importar <code>streamlit</code>	2,1
cargar el modelo SPECTER	2,5
barrido DBCV (elegir la granularidad)	2,3
leer los datos del disco	0,2
TOTAL	52,2

Concepto: Lo que enseña la tabla

44,5 de los 52,2 segundos son *importar librerías*. Y la pantalla de inicio de Froth (el título, el desplegable de temas, la caja para pegar un título nuevo) **no usa ninguna de ellas**. Estabas esperando 45 segundos por código que no se iba a ejecutar.

Primer arreglo: importar tarde

Antes, arriba del todo de `froth/cluster.py`:

```
import umap
import hdbscan
```

Después, dentro de la función que de verdad lo usa:

```
def reduce_to_2d(vectors):
    import umap          # se paga aqui, no al abrir la app
    reducer = umap.UMAP(n_components=2, metric="cosine", random_state=42)
    return reducer.fit_transform(vectors)
```

Ojo / error típico

El coste no desaparece: se mueve. UMAP sigue tardando sus 30 segundos. La diferencia es *cuándo*. Antes lo pagabas mirando una pantalla en blanco, sin saber si el programa estaba vivo. Ahora lo pagas cuando cosechas un tema nuevo, con una barra de progreso delante que te dice qué está haciendo. La misma espera se siente completamente distinta según si sabes por qué esperas.

Pero faltaba lo gordo, y estaba escondido

Con los imports arreglados, el arranque medido en frío bajó a 8,8 segundos. Y sin embargo, al abrir la app de verdad, **seguía tardando más de tres minutos y medio**.

La lección está justo aquí: **el cronómetro solo mide lo que le pides que mida**. Yo había medido los imports, no la app entera. Al mirar lo que la app imprimía mientras trabajaba apareció el culpable de verdad:

Naming the subtopics (reading abstracts)...

Poner nombre a los 44 subtemas. Para cada uno hay que leer sus abstracts, sacar frases candidatas, convertirlas en vectores con SPECTER y quedarse con la que mejor representa al grupo (es el apunte 40). Eso es caro. Y se hacía **entero, cada vez que abrías la aplicación**.

Concepto: Por qué se repetía si ya estaba “cacheado”

El código ya usaba `@st.cache_data`, que guarda el resultado **en memoria RAM**. Funciona de maravilla mientras la app está abierta: cambias de pestaña y no recalcula. Pero al cerrar la app, esa memoria se libera. Al día siguiente vuelves a pagar los tres minutos. Una caché en RAM te protege de repetir dentro de una sesión. No te protege de repetir *entre* sesiones. Para eso hace falta el disco.

Segundo arreglo: guardar en disco, con la clave correcta

Los nombres se guardan ahora en un archivo `.json`. La parte interesante no es guardar: es **con qué etiqueta**.

```
key = f"{version}|{mcs}|{years[0]}-{years[1]}"
digest = hashlib.sha1(key.encode()).hexdigest()[:12]
# -> 2_Datos/processed/subtitles_36e4d383cfce.json
```

La etiqueta mezcla las tres cosas de las que dependen los nombres:

- **la versión de los datos:** si cosechas más papers, los subtemas cambian;
- **la granularidad:** mover el deslizador crea otros subtemas;
- **el rango de años:** filtrar por años deja fuera papers, y los grupos cambian.

Si mueves cualquiera de las tres, la etiqueta cambia, el archivo guardado no aparece, y se recalcula. Si no mueves ninguna, se lee del disco al instante.

Ojo / error típico

Este es el error clásico de las cachés, y da miedo porque no falla ruidosamente. Si la clave se te queda corta (por ejemplo, solo el nombre del corpus), el programa te enseña los nombres viejos sobre datos nuevos. No se rompe, no avisa: simplemente te miente. Una caché con la clave mal puesta es peor que no tener caché.

Concepto: Esto NO contradice la regla 9

Podría parecer que guardar el resultado es lo contrario de “recalcular para cada corpus”. No lo es. Se sigue calculando para *este* corpus, con *sus* datos. Lo único que se elimina es calcular **dos veces la misma pregunta**. Un valor por defecto sería no medir nunca; esto es medir una vez y acordarse.

El resultado

	Antes	Ahora
Abrir la app hasta poder usarla	>3,5 min	≈30 s
Solo cargar librerías	52,2 s	8,8 s
¿Carga SPECTER al abrir?	sí	no

Pruébalo tú (teclado en mano)

Compruébalo tú, y de paso aprende a medir:

1. Abre Froth con el acceso directo del escritorio y cuenta mentalmente hasta que veas la pantalla de inicio.
2. Ahora mueve el deslizador de granularidad a un número que no hayas usado nunca (por ejemplo 7). Fíjate: *ahí* sí espera, porque esa combinación no está guardada todavía.
3. Vuelve a moverlo a 18 y luego otra vez a 7. La segunda vez es instantánea: ya está en el disco.
4. Mira la carpeta `2_Datos\processed\`. Verás un archivo `subtitles_...json` por cada combinación que hayas probado. Ábrelo con el Bloc de notas: son los nombres de tus subtemas.

En una frase

Si un programa tarda en abrir, mide antes de tocar: casi siempre está haciendo trabajo que no hace falta todavía (impórtalo tarde) o trabajo que ya hizo ayer (guárdalo en disco, con una clave que incluya TODO lo que puede cambiar el resultado).